



Lesson planner to relieve pressure on Primary and Secondary School teachers.

Koda Allison (220294070)

May 2025

BSc Computer Science (Software Engineering)

Supervisor – Laura Heels

Word Count: 14725

Abstract

Existing research and teacher feedback from my own studies showed lesson planning is one of the most time-consuming tasks that make up a teacher's workload. It is repetitive, takes time away from actual teaching and adds pressure outside of traditional working hours, contributing to a poor work life balance.

To address this, I made use of advances in generative AI to develop a web application capable of generating lesson plans. The system uses OpenAI's API to produce a lesson plan based on teacher inputs such as year group, subject and brief lesson objectives. Built using Next.JS, Prisma and PostgreSQL with authentication provided by Google via NextAuth, the app is modern and user-friendly. Allowing teachers to generate, edit, save, download and search through their plans within one platform. Prompt design was a key part of this project and was critical to the quality of lesson plans that would be generated. Good prompt design was ensured by research into prompt engineering techniques and a teacher survey.

The result was an application that can generate a structured lesson plan in under 30 seconds.

Although there is room for improvement, it still offers significant time saving benefits compared to writing plans out manually. The finished system also followed HCI principles, including Nielsen's heuristics, to ensure a good user experience and a system that was usable.

Acknowledgements

I would like to thank my supervisor, Laura Heels, for her guidance and support over the course of this project.

I would also like to thank the teacher's that took part in my survey and the fellow student who helped evaluate my application.

Declaration

I declare that this dissertation represents my own work except where otherwise stated.

Table Of Contents

Abstract.....	2
Acknowledgements.....	2
Declaration	2
List of Tables	5
List Of Figures	5
1 Introduction.....	6
1.1 Context & Motivation.....	6
1.2 Aim and Objectives	6
1.3 Dissertation Outline	7
1.3.1 Changes from proposal	7
1.3.2 Project Structure	8
2 Background Review	9
2.1 Justification for Solution	9
2.2 Prompt Engineering.....	11
2.3 Related Tools and Tech Stack.....	11
3 What was done, and how.....	13
3.1 Requirements.....	13
3.2 User Study	14
3.3 Prompt Engineering.....	15
3.4 Database Design & Implementation.....	18
3.5 Authentication	20
3.6 UI/UX Design	20
3.7 Integration with OpenAI	25
3.8 Feature Development.....	26
3.8.1 Landing Page.....	26
3.8.3 Dashboard	28
3.8.4 Editing Existing Plans	29
3.9 Testing	29
3.9.1 Unit Testing.....	29
3.9.2 Manual Feature Testing.....	30
3.10 Summary of Development and Tech Stack.....	33

3.10.1 Coding Standards.....	34
3.10.2 Technologies Used.....	35
4 Results and Evaluation	36
4.1 Functional Requirements Review	36
4.2 Non-Functional Requirements Review.....	38
4.3 Performance.....	38
4.4 Usability	39
4.4.1 Lesson Plan Usability and Customisation	39
4.4.2 Application Usability.....	39
4.5 Limitations.....	42
4.6 Overall Effectiveness	44
5 Conclusion	44
5.1 Summary.....	44
5.2 Evaluation of Aims and Objectives.....	44
5.3 Future Work and Expansion	46
5.3.1. Fundamental Improvements	46
5.3.2 Future Expansion.....	47
5.4 Final Reflection.....	47
References	48
Appendices	50

List of Tables

Table 1 – Requirements.....	13
Table 2 – Manual Testing Results.....	30
Table 3 – Functional Requirements Evaluation	36
Table 4 – Non-Functional Requirements Evaluation.....	38
Table 5 - Think Aloud Task Observations	40

List Of Figures

Figure 1: Project Timeline Gantt Chart.....	8
Figure 2: E-R Diagram.....	19
Figure 3: Landing Page Figma Designs	22
Figure 4: Dashboard Screen Figma Design.....	23
Figure 5: Plan Generator Screen Figma Design	24
Figure 6: Plan Editor Figma Design.....	25
Figure 7: Form Validation Schema.....	26
Figure 8: Initial Generator Implementation	27
Figure 9: Final Generator Implementation.....	28
Figure 10: Test Suite.....	29
Figure 11: Landing Page	31
Figure 12: Google OAuth Login.....	31
Figure 13: Dashboard Screen After Successful Authentication	32
Figure 14: Generator form screen.....	32
Figure 15: Loading State shown while generating plan	33
Figure 16: Complete Lesson Plan Preview with Network.....	33
Figure 17: Commit History Snippet.....	34
Figure 18: Downloaded lesson plan output	36
Figure 19: Lesson Plan Preview.....	37
Figure 20: Loading State shown on Dashboard.....	41
Figure 21: Error Correction Message for missing field.....	42
Figure 22: Error Correction Message for invalid field	42
Figure 23: Edit Page.....	43
Figure 24: Download Confirmation.....	43

1 Introduction

1.1 Context & Motivation

Teachers are an important part of society, playing a vital role in educating and shaping future generations. However, they are often overworked and underappreciated, with lesson planning and resource creation consuming a significant portion of their time. According to an Ofsted report, teachers work an average of 50 hours per week, which is substantially higher than the standard workweek of other professions. What makes this worse is that less than half of their time is dedicated to actual teaching [1]. The remainder is spent on administrative tasks such as lesson planning, grading, and resource preparation.

Although teachers find their profession rewarding there is a “crisis in teacher recruitment and retention” [2] and this is likely because of the reported poor work-life balance due to excessive workload. “For many current teachers, excessive workloads, rather than pay, are the primary factor pushing them to leave the profession.” [2] A key contributor to the high workload is that lesson planning is done manually, requiring teachers to sift through old plans, create new ones from scratch, or rely on scattered resources. The lack of a structured, centralized solution leads to inefficiencies and results in teachers spending excessive time on tedious, non-teaching activities. These factors combine to not only lead to high workload/poor work life balance but also effect overall quality of education given to students.

With this dissertation project, I aim to investigate the current methods used by teachers for planning lessons by conducting a survey. I will analyse the results of the survey to identify common strategies and components that are considered essential by teachers – such as timings and objectives. I will combine this with research into prompt engineering techniques such as assigning a role and prompt structure to come up with a suitable prompt that takes advantage of the capabilities of generative AI.

Teachers will be able to access these lesson plans through a web application built mainly using NextJS. They will be able to adjust prompts to tailor certain aspects of a plan such as age group, subject area, etc. Alongside this they will be able to store lesson plans for quicker retrieval later and download them as to be able to access them offline or share them with other teachers.

1.2 Aim and Objectives

With this project, I aim to reduce the workload for teachers and consequently relieve the pressure on them while still maintaining a good standard of teaching. I will do this by creating a web app using NextJS and OpenAI amongst other technologies to generate lesson plans, This will allow teacher to spend less time on tedious tasks, improving their work-life balance and allowing them to give more attention to students.

Objectives:

1. **Research Prompt Engineering:** Gain knowledge on prompt engineering techniques to generate good plans that are effective and usable. Usability being defined as being easy to follow, clearly outlining the timings and parts of the lesson.

2. **Conduct a teacher survey:** Gather insights to investigate the current process teachers have for lesson planning, aiming for at least five responses. Analyse common methods/themes across their responses.
3. **Develop structured AI prompts:** Use the results of the survey to help with coming up with a prompt. Ensure it generate suitable, good-quality lesson plans. By this definition, a plan should be easy to follow, clearly outline tasks, and be relevant to age/subject.
4. **Create a requirements table:** Provide at least 10 milestones to help structure development of the application. Include priorities to help with risk mitigation and time management.
5. **Create a high-fidelity prototype of the User Interface:** Create wireframes then translate these to Figma to come up with a user-friendly UI that follows Nielsen's heuristics.
6. **Implement backend API:** Develop API endpoints to interact with Open AI's API to allow generation of lesson plans and integrate with a database to store and retrieve generated lesson plans.
7. **Develop the web application:** Allow teachers to interact with the program. This web app should be easy to use and have the basic CRUD operations implemented.
8. **Integrate working Authentication** – Use Next-Auth to handle secure user registration and login, also help to uniquely identify lesson plans in the back end.
9. **Evaluate application:** Complete manual testing to check if lesson plans can be generated using the UI and that lesson plans are usable. This means objectives, timings, and parts of the lesson are clearly defined. Also carry out unit testing to verify the reliability of the application.
10. **Evaluate effectiveness:** Measure time saved by using the application compared to current methods.

1.3 Dissertation Outline

1.3.1 Changes from proposal

I have slightly changed the tech stack I will be using as I think the change will speed up development of the application and avoid unnecessary complexities. Initially, I proposed using a separate backend (flask). However, after looking deeper into the capabilities of NextJS I realised that I could make use of their in-built API router for my endpoints so there was no need for the separate flask backend. This way of working should be more efficient but also still be effective and allow me to meet my objectives.

I've also decided to add authentication using Next-Auth. I realised it's important for teachers to securely log in, especially since they'll be storing and retrieving their own lesson plans. Next-Auth is popular as it is easy to integrate and provides robust security. This addition allows me to make plans uniquely identifiable in the backend. This means that lesson plans can easily be retrieved, organised, and managed per user, significantly improving usability and overall data security. Lastly, I have also decided I will come up with a requirements table. I made this decision after looking back at old work from past modules I remembered how helpful it is for planning a project. A requirements table allows me to track my progress, give me clear objectives and provides a basis

for evaluation at the end of the project as to how well I met the requirements. Alongside this, I refined my prototyping process. I will now use paper sketches to come up with a low-fidelity prototype focusing more on idea generation. I will then translate the paper prototype into Figma where I will focus more on key HCI concepts.

1.3.2 Project Structure

I chose to split my project up into two broad phases, this will allow me to be more flexible in the work that I am completing and not have too strict of a schedule but did create a Gantt chart (Figure 1) to provide a rough guide. The phases describe the focus of tasks at the time:

- Phase one is all about getting a thorough understanding of the problem and starting to sketch a solution.
- The second phase builds off the first. Focuses on implementing the solution

This will help me to mitigate risks and fit into my busy schedule. For example, if survey responses take longer than I expect I can still work on another task such as refining my prompt or thinking about sketching the UI to not waste time and keep making progress.

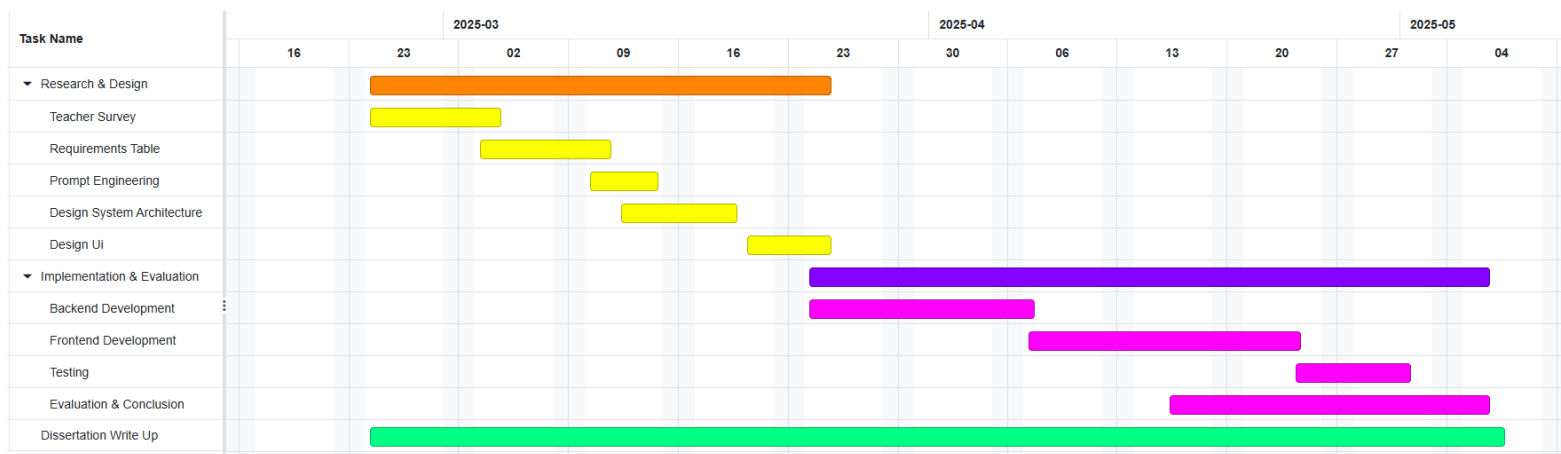


Figure 1: Project Timeline Gantt Chart

I will use the Gantt chart to track overall progress and ensure I am on schedule. To focus on short-term goals I created a Kanban board on notion (Appendix A). This board breaks down tasks from the gantt chart and requirements table making them smaller and more manageable. Each task is categorised into “To Do”, “In Progress” or “Complete” to stay on top of progress.

Phase One - Research & Design.

Teacher Survey, AI Prompt Engineering, Requirement Table, User Interface Prototyping, System Architecture Plan.

I will start by conducting a *teacher survey* to collect info on current lesson planning methods. Using their results I will generate prompts for generative AI. These prompts will be designed with latest *prompt engineering* techniques in mind, so they are of the highest quality and are a good structure that is easy to use

I will also create a detailed *Requirements Table* to define projects requirements alongside the priority of each requirement which will help me with project management. I will use this requirement table when designing *User Interface (UI) Prototypes*. To design the UI I will first come up with low fidelity prototypes. I will use paper prototyping techniques such as crazy-eight to focus on idea generation over quality of the design. I'll use this to pick out my best ideas and turn them into a detailed design on Figma that follows Nielsen's usability heuristics [20]. However, to avoid spending too much time on this phase I will not make a full prototype that includes the user journey. Since I am the only developer, I will just make sure I understand of what I intended the flow to be.

The final thing I plan to do in this part of the project is to design the *System Architecture*. I will create a UML diagram to plan out the database schema. This will provide me with a blueprint to follow during development and ensure the program is maintainable.

Phase Two – Implementation & Evaluation

Backend Development, Frontend Development, Testing, Evaluation & Conclusion

The *backend development* of the application will involve setting up the API routes to OpenAI API in NextJS alongside building and integrating the database. I will focus on this first and then move to *frontend development* which will involve building the UI based off the Figma designs and then incorporating authentication into both front and back end using NextAuth. This will ensure lesson plans are uniquely identifiable and that their access is restricted to the teacher who generated it. I will then move on to *testing* and the *Evaluation & Conclusion*. These go hand in hand, and I will use the results of testing the security, functionality and usability of the program in my evaluation. Also, in my evaluation I will look at the effectiveness of the application measuring the time it takes to come up with lesson plans with traditional methods and comparing that to using my application.

2 Background Review

2.1 Justification for Solution

The workload burden on teachers is becoming more documented. Ofsted's report on teacher well-being in schools' [1] highlights that many teachers are working 50+ hours a week – significantly higher than national averages. What makes this worse is that less than half of their time is spent on teaching. Instead, lesson planning, marking, admin and other tedious tasks make up a large portion of their time. This creates stress and inefficiencies which means a lot of teachers suffer from burnout. Ofsted is a very credible source, and their report highlights the impact of inefficiencies within teaching. Along with the impact on the workload and consequently work-life balance of teachers. However, there are no solutions provided in this report, and this is a gap that I aim to fill with this project. The issue of workload is also influencing recruitment and retention of teachers. This is shown in a report from the UK government which mentions that “excessive workloads rather than pay, are the primary factor pushing teachers out of the profession” [2] This shows how that if this project can be successful in streamlining/automating certain aspects of the job it could help retain teachers and improve their work-life balance.

Luckin et al. [21] argue that AI has strong potential to reduce teacher workload, especially when it's designed to support rather than replace teachers. Their paper, published by Pearson, is a credible academic source and directly supports my aim of reducing time spent on tasks like lesson planning. It highlights how AI can help with repetitive tasks but keep the teacher in control. Something I will try to implement as I am planning to allow users to edit the generated plans to suit their own teaching styles and students.

There is some research surrounding solutions. One study from Powell and Courchesne's [3] explores the use of ChatGPT to generate first-grade science lesson plans. They found that the AI could produce structured plans with clear outlines, but they found that there were some accuracy issues. This is relevant as it validates the concept of using AI in teaching but, it also shows limitations. This study is peer-reviewed which gives me confidence in its reliability. Allowing me to use their findings to influence my project's emphasis on careful prompt design and teacher-driven inputs to hopefully come up with better lesson plans. A study from Kamran and Goksu [4] also generated lesson plans via ChatGPT, but they went even further testing them in the classroom. This is a reliable source which is also peer reviewed as it is published in the International Journal of Technology in Education. This ensures validity in the study and what they found: some improvement in results from using AI lesson plans but not significant. However, the more important part to take away from their study is that there was no negative effect on learning and AI generated plans can be as effective as human ones. This gives confidence in the usefulness of the tool I am planning to create.

An aspect of generating plans is understanding the students; something that the AI will likely not be as good at as the teachers. The paper "Tailoring Education with GenAI" [5] is a preprint so is slightly less credible than other sources but still useful. It presents a lesson planning system that adapts content based on student demographics and learning goals. It goes beyond content generation and introduces personalisation. Importantly, the paper evaluates its system through quantitative (time saved) and qualitative (teacher satisfaction) metrics. This methodology evaluation is something I will look to use for my solution, comparing time saved and plan usability in my own system. While I won't be adding student-level personalisation in this version, it is a good idea for future development.

Another relevant study by Corp and Reville [6] looked at how student teachers used ChatGPT to support lesson planning. This source comes from The Texas Forum of Teacher Education which is a peer reviewed journal. It is somewhat credible and offers valid insight into classroom experiences. They found that while the AI helped speed up the process and offered useful structures, there were clear risks involved. These involved over-reliance on technology and concerns over accuracy. Users appreciated having a framework to start from but had to remain vigilant about the content's quality and relevance. This study reinforces the importance of designing systems that support the teacher rather than replacing their judgment.

2.2 Prompt Engineering

A key part this project is prompt engineering, how well a prompt is written is reflected in the response. This means the quality of a prompt directly affects how useful the generated lesson plan will be. To get quality lesson plans, prompts need to be detailed and specific. Therefore, I made sure I spent time researching common practices and evaluated multiple sources to decide what prompts and prompt structure will work best for this project.

I previously mentioned Powell and Courchesne [3] in justifying the need for my solution. This is a credible source due to its academic peer review, and it mirrors my goal. The key point for me from this article in relation to prompt engineering is that weak prompts would have a strong impact on quality of output and therefore how good my application is. This made me decide to use template-based prompt structures that guide the AI to include things like objectives, timings, and activity stages. While their work is based on U.S. curriculum, the concept of structure is still highly relevant.

“Generative Artificial Intelligence for Teaching, Research and Learning” from UC Davis [7], isn’t peer-reviewed but comes from a reputable academic institution and focuses specifically on generative AI best practices in educational environments. It has a section specifically on prompt engineering that highlights the need for iterative refinement. This involves iteratively testing and tweaking prompts based on output quality. The source may not be formally validated but, it’s still offers good insight for my project and prompt design.

The GIANT AI Guide [8] introduces the concept of few-shot prompting. This involves providing sample inputs in the prompt to guide the AI’s behaviour. This source is not academic, so it lacks formal validation, but it is still relevant. Few-shot prompting is widely used in real-world applications, I could incorporate this by embedding example lesson structures built from survey responses directly into the prompt. This will mean a consistent output that is usable and contains all necessary aspects of a lesson plan.

Schulhoff et al. [9] explored prompting strategies such as ReAct (Reason + Act), combining step-by-step logic with actionable outputs. It is not yet peer-reviewed but is cited a lot which gives me confidence in what it is saying about current strategies for prompt engineering. While slightly more advanced than needed for my current product, it was good to consider and will be valuable especially for future work. Especially for features like generating a full week of lessons or adapting plans to a specific teaching style or curriculum. Despite being a preprint, the authors are credible researchers, and the paper is widely cited, making it valid.

Together, these sources helped guide the design of my prompt. Influencing my use of structure, role assignment and iterative refinement to ensure lesson plans were good quality. This is discussed further in Section 3.3.

2.3 Related Tools and Tech Stack

The quality of prompt is crucial but is useless without a good system for teachers to interact with. So I reviewed existing tools and what technologies could be used.

TeachMate AI

TeachMate AI [10] is a commercial tool offering AI support for education. While it shows the direction the industry is moving in, it falls short in key areas such as centralised lesson storage, customisation, and has free-tier feature limitations. This gives my project a clear opportunity — to deliver a solution that offers an application that can allow the saving and editing of lesson plans.

Twinkl

Twinkl [11] is a popular, well-established education resource platform. It offers templates, worksheets, and ready-made plans that can be adapted. It has started to integrate AI into some of its tool; these tools are improving but most of the platform's value comes from human-created content. Access to most content is locked behind a paid subscription, and the AI tools are lacking. For lesson planning, Twinkl's approach is more traditional and lacks the flexibility of tools built around AI from the ground up.

Brisk Teaching

Brisk [12] is a lightweight platform that integrates AI tools into existing teaching workflows. It allows for quick generation of feedback and plans. While it prioritises simplicity and accessibility, it doesn't include lesson storage, categorisation, or transparent structuring. Its output still requires heavy teacher review and rework in many cases, and its tools are limited in scope compared to what teachers may need for full lesson planning.

From these tools there are a few gaps in Ai teaching tools that my project aims to fill. These tools aim for one off lesson plan generation and lack ability to customise the plans inside the platform and also do not offer saving into one repository like I intend to. This means my project can offer an even more time efficient solution then what currently exists.

This project originally proposed a Flask backend with a PostgreSQL database. Flask is lightweight and has been used in multiple of my modules already. It can be ideal for quick API prototyping. However, after further research into deployment, authentication, and integration with OpenAI, I decided to move to a modern JavaScript stack.

Several popular AI-powered education tools - including TeachMateAI - use JavaScript-based frontends. Likewise, Next.js offers a clear advantage in terms of built-in API routing and seamless integration with frontend components. Using the App Router, I can manage both frontend and backend logic in one codebase while ensuring secure communication with the OpenAI API.

The decision to use Next.js with Prisma and PostgreSQL allows for faster development and easier scaling, especially with tools like NextAuth handling authentication and Tailwind CSS ensuring responsiveness. Flask remains a valid option, especially for smaller or Python-heavy projects. However, the combination of OpenAI's Node.js client library and unified frontend-backend made Next.js a more practical and maintainable choice for this system. This decision was influenced by

the need for speed of development, possible scalability, maintainability, and smooth user experience.

3 What was done, and how

3.1 Requirements

Table 1 – Requirements

Requirement	Description	Rationale	Priority
FR1	User can log in using google OAuth	Need to link the lesson plans to who created it so each user should have a unique account.	High
FR2	System must generate lesson plans based off AI prompt	Automates planning of lessons saving teachers time and reducing workload	High
FR3	System must allow for teachers to edit lesson plans	Allow the plans to be tailored to teachers' needs. As they will know and understand their pupils	Medium
FR4	System must be able to store and retrieve past lesson plans	Gives centralised access allowing for plans to be easily reused.	High
FR5	System must allow user to be able to export lesson plans as PDF	Allows for offline access and easily sending it to and collaborating with other teachers	Low
FR6	System must allow users to search stored lesson plans	Improves user experience makes retrieval more efficient	High
FR7	System must have a dashboard for managing lesson plans	Allow easy management of plans.	Medium
NFR1	System must follow Nielsens Heuristics	Good metric for ensuring a good use experience	Medium
NFR2	AI generated responses must have fast turnaround time	Ensure wait times aren't long to fit with purpose of the application and ensure positive user experience	Medium
NFR3	System must be secure with no unauthorised access to lesson plans	Prevents leaks of lesson plans that potentially have sensitive data	High

3.2 User Study

I conducted a survey by emailing teachers that were family friends and that worked at my previous schools. This was done to gather insight into the way teachers currently plan so my solution could follow the same principles. Each question was designed to gain specific insight whether that be further validation of the need for my application or what is essential to a good lesson plan. I ensured that before the participant answered questions that they were shown a briefing of the project and the survey. This brief included the aims and objectives of the project, why their response was needed and made sure they were aware that it was anonymous and if they wanted to they could opt out at any time and not answer a question if they did not want to.

Due to this being an individual project along with time/ethical considerations, I only gained 6 responses. However, this still provided useful insight from actual teachers that I have implemented into my application.

Question 1: How long do you typically spend per week on planning lessons?

The purpose of this question was to understand the workload further at a more local, relatable level and ease the teacher into the survey.

The findings from this question were that the majority (around 75%) said 1-4 with some stating 4-8. This confirmed the need for an application that streamlines the process of planning lessons aligning with the Ofsted report [1] previously discussed

Question 2: Do you feel you spend too much time on tedious tasks such as lesson planning leading to a negative impact on work-life balance and/or student engagement?

The purpose of this question was to validate the necessity of the application.

The findings were that 83% of participants felt that their work-life balance and student engagement is in fact impacted negatively by tedious tasks such as lesson planning. Reinforcing the potential value the application holds.

Question 3: What is the most frustrating part of lesson planning?

The purpose of this question was to pick out areas of frustration which can be a priority to address within the application. Key frustrations included:

- generating engaging content,
- repetition
- adapting to changes in curriculum

This can be solved by the application as AI can reduce repetition, it can come up with a lot of ideas that could be engaging. It can search the web quicker/scan documents quicker to adjust to any changes in curriculum.

Question 4: Briefly describe how you usually plan lessons?

The purpose of this question was to understand the current process more and see what areas of this can be translated into the user experience or prompt design.

I found that teachers typically begin with outcomes, structure tasks and some spoke about integrating long term plans. This helped me decide what parts of the prompt can be modifiable as well as highlight that generated lesson plans should include outcomes and tasks.

Question 5: List 3 things a lesson plan must include to be useful

The purpose of this question was to determine what components are essential for creating a good lesson plan that can be thought of as effective and practical.

The common themes across the responses were,

- Clear learning objectives (100% of respondents).
- Precise activity timings (over 80%).
- Effective differentiation strategies (over 50%).

Engagement methods was also cited by multiple teachers. These insights directly shaped the structure of my prompt and design of my application. They also offer a good metric for my evaluation.

Question 6 & 7: Are you in favour of using AI to generate lesson plans? & Would you find this app useful

The purpose of these questions was to gauge the perceived acceptance and usefulness of generating lesson plans using AI. Most responses were “Yes” or “Maybe” which is promising.

3.3 Prompt Engineering

Designing a prompt that generates a high-quality lesson plan is the “make or break” of this project. Generative AI can generate large amounts of practically anything, the quality of what is produced is dependent on the prompt. To ensure good prompts that generate a high-quality output I made use of both the literature on prompt engineering (Section 2.2) and the findings from my teacher survey (Section 3.2). Rather than using a simple, generic prompt such as “generate me a lesson plan”, I applied a series of prompt engineering techniques.

Technique 1: Role Assignment

The prompt starts with: *“You are a highly experienced teacher creating a detailed lesson plan.”*. According to the UC Davis guide [7], assigning a role gives the AI context, which improves relevance and tone. The GIANT AI Guide [8] also highlights this as a best practice for generating professional outputs. The teachers in my survey said that engaging content and structured outcomes were hard to generate - so telling the AI up to “think like a teacher” helped mimic their professional tone and structure.

Technique 2: Structure & Formatting

The responses from Q4 and Q5 in my survey highlighted key components that a teacher would include in a lesson plan. I used this to structure the prompt and the output I am requesting within it. It clearly lays out all required sections of the plan:

Learning Objectives, Introduction, Activities (with timings), Differentiation, Engagement Strategy, Extension Tasks, Resources, Conclusion

This approach keeps in mind literature from my background review, following Powell and Courchesne [3], who showed that under-specified prompts often lead to missing or incomplete sections. Also, the UC Davis guide [7] recommends being specific about content, format and structure. By explicitly defining the layout and listing each part, I ensured that no critical content was left out. This means the generated lesson plan should be good quality and usable.

Technique 3: Few-Shot Prompting/Scaffolding

While I did not include actual “few-shot prompting” examples (e.g. full sample lesson plans), I still made sure that I applied the concept. I also followed guidance from the GIANT AI Guide [8], which recommends structured scaffolding when few-shot examples are impractical.

I did this by creating a clearly structured, section-based prompt that listed each part of the plan. This gave the AI guidance meaning it will produce consistent outputs.

Technique 4: Iterative Refinement

Both UC Davis [7] and Schulhoff et al. [9] recommend refining prompts over time and experimenting with slightly different formats/wording. To put this into practice, I made use of the ChatGPT interface to ensure that the prompt I will implement in my application will produce a good lesson plan. I did this to avoid wasting OpenAI tokens but get feedback from the same AI model.

Prompt 1 looked like this:

Generate a lesson plan for a Year 6 History class. The lesson should last 60 minutes and include learning objectives, an introduction, some activities, and a conclusion.

This version was somewhat functional, it had a basic structure and mentioned key sections that were found to be necessary in the user study. However, it is too vague and didn’t fully apply the prompt engineering techniques. The role of a teacher was implied but not actually specified, the result didn’t always include specific timings as there was only one mention of the timeframe. There was also no guarantee that it would include differentiation meaning consistent output. From this I knew I needed to implement more structure and guidance.

Using what I learned from Prompt 1, I tested Prompt 2:

You are a teacher creating a lesson plan. The topic is "[Lesson Topic]" for "[Year Group]" and it should last "[Lesson Duration]".

Please include:

- Learning Objectives*
- Introduction (with timings)*
- Main Activities*
- Conclusion*

This version had some basic structure. It was clearer than the first, outputs started following a more consistent format. However, I noticed that timing still wasn't always included.

This final version used everything learned from the testing, literature and survey: Role prompting, Structure, Specific instructions, Scaffolded design. I also started thinking more about how the response to the prompt would be passed in the web application and what the response would look like. This led to adding instructions stating to have clear subheadings with "##" making the content easy to parse and separate.

The exact prompt used in the program:

You are a highly experienced teacher creating a detailed, digital-first lesson plan for classroom use.

The lesson topic is: "\${data.title}"

The subject is: "\${data.subject}"

The target group is: "\${data.ageGroup}"

The total lesson duration is: "\${data.duration}" minutes

The learning objective is: "\${data.objective}"

\${data.style} ? `The teacher prefers the following teaching approach: "\${data.style}"` : ``

\${data.notes} ? `Consider the following additional context or classroom adaptations:

"\${data.notes}"` : ``

Please generate a structured lesson plan that includes the following sections:

- 1. Learning Objectives – Clearly stated, measurable outcomes.*
- 2. Introduction – An engaging starter activity (include duration).*
- 3. Main Activities – Sequential, clearly detailed activities with precise timings.*
- 4. Differentiation Strategies – At least two explicit methods to support varied pupil needs, including support for students with Special Educational Needs (SEND).*
- 5. Engagement Strategy – One interactive, student-led or hands-on approach.*
- 6. Extension Tasks – Challenging activities for advanced learners or early finishers.*

7. Digital Teaching Resources – Generate full content for any worksheets, quiz questions, diagrams, or reflection tasks

8. Conclusion – A short reflection or wrap-up activity with timing.

Instructions:

- *Ensure all sections include specific timings that total exactly $\${data.duration}$ minutes.*
- *Use a formal and concise tone suitable for professional educators.*
- *Ensure all resources are fully digital — no printing or physical materials.*
- *Format using clear ## Section headings for display in a web app.*

By making small improvements and testing each version for output quality, I was able to design a prompt that consistently produces structured, usable lesson plans suitable for real teachers.

3.4 Database Design & Implementation

The first thing I did when thinking about the backend aspects of the application was to design an Entity-Relationship (ER) diagram. This diagram allowed me to visualise how the users and lesson plans would be related to each other alongside making sure the authentication side was handled correctly. Each lesson plan is linked to a specific user via a foreign key (userId) in the LessonPlan table. This ensures that lesson data can be protected from unauthorised access helping to satisfy NFR3. When querying lesson plans, I ensured I included the current user's ID and that this matched the ID linked to the lesson plan. This ensures that users can only view or edit their own lesson plans, even if they attempt to access another user's plan by ID.

Having this diagram meant that when I started development it was easy to implement the database as there was already a plan in place; I wasn't constantly adjusting things to work correctly which would slow down the process.

The diagram (Figure 2) outlines the four tables of my database: User, LessonPlan, Account and Session. Each user can have many lesson plans, and this is shown by the one-to-many relationship. The Account and Session tables are included because they are required by NextAuth. My schema design followed best practices, defining relationships in a way that makes it reliable and maintainable. [13] I applied concepts learned in other modules and forums such as Stack Overflow, alongside established guidelines for clean E-R layouts and schema design [14]. Data types, primary keys, and foreign key constraints were clearly specified.

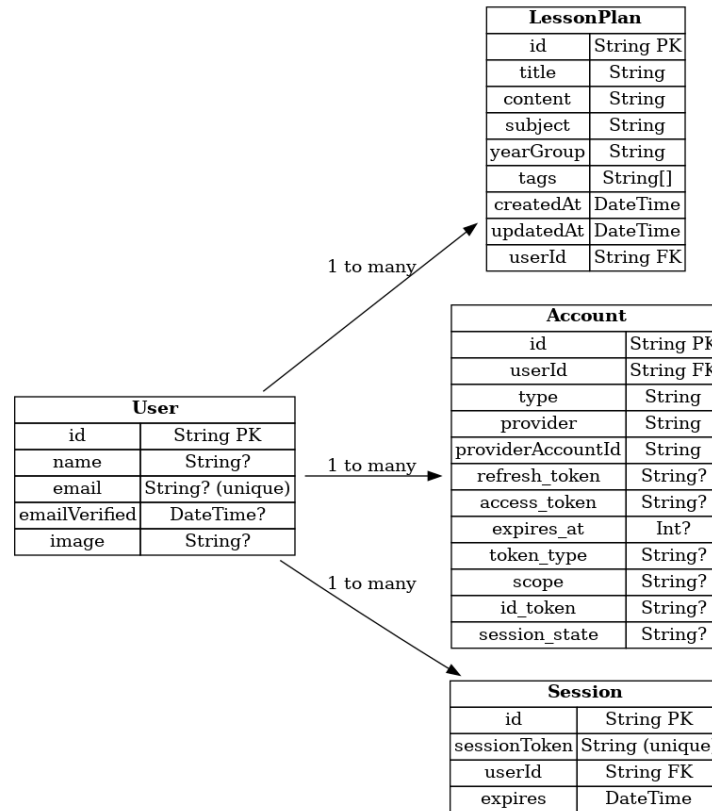


Figure 2: E-R Diagram

Using the diagram as a guide I implemented this database using PostgreSQL which was hosted on Neon via Vercel. When deployed, the front and back end is manageable on one platform. [16] This means that the application is easily maintainable and scalable. I used Prisma ORM to define the schema within the application and then used the migrate feature to ensure the live database and schema in the application were synced. [15]

I considered a few different solutions when designing the database. I ended up choosing to use Prisma as I knew I wanted to use Next.JS and Prisma is easy to integrate into Next.JS/NextAuth. I did consider Firebase as I had heard that offered good free plans but upon further inspection of the tool decided against it. I ruled it out as it offers NoSQL models which does not fit the structured, relational database that I designed in the E-R diagram. I also thought about using Flask and SQLAlchemy which I have used previously in university. However, flask does not integrate as well with Next.JS and modern JavaScript tech stacks and having this modern front end was more important to me so I could easily develop a clean, responsive front end. NextAuth also provides easy functions to get the current server session which let me easily implement protection of pages. If the user had no session, they would be redirected to the landing page and not have access to the system.

3.5 Authentication

To enable users to log in and to link saved lesson plans to their account, I used a tool called NextAuth.js combined with Google OAuth [17]. This meant that my project would meet FR1. I was able to quickly and easily provide a secure log in function within the application without having to develop an authentication system from scratch. NextAuth offers support for many different identity providers, but I choose to use google as most teachers will have an account - whether that be personal or through their school - therefore the login process will be efficient and seamless, they will not have to create an account.

To use Google OAuth, I had to set up a project using the Google Cloud Console. I registered my project using localhost:3000 URLs - as I would only be testing on a local machine environment - and the redirect URL provided by next auth (/API/auth/callback/google). Then google gave me a client ID and client Secret to add to my .env file that ensured my application was secure.

NextAuth has a Prisma Adapter [18] which automatically links user sessions and accounts to my database. This meant I did not have to write any custom user management code. When a teacher signs in for the first time their details are stored. Every session is logged and to prevent unauthorised access to lesson data I used the get server session functions offered from NextAuth. I ensured there was an active session for all routes that started /Lesson this included the dashboard page, edit and generation page. So without a session the user cannot access any functions of the application. Instead, they are redirected to the landing page and forced to log in.

Lesson plans are linked to the user that is logged in they are created via the userId foreign key in the database. This meant that when fetching lesson plans, I ensured to add "where: { userId: session.user.id }" to ensure they were only accessing plans they owned.

By using google OAuth I did not have to worry about password storage, encryption/hashing, login verification and having account recovery methods. Adding these features manually would have increased development time, added complexity and added security risk. Furthermore, Google has its own multi factor authentication which I was able to take advantage of, adding security without any extra work.

3.6 UI/UX Design

After I established the logic of the backend, I was able to better understand how prompts would need to be modified by users, I combined this with what I learned in the User Study (Section 3.2) when designing the UI and UX. Since the aim of the application is to speed up lesson planning it was important that the interface required as little learning time as possible. I tried to ensure the design was simple, clear and practical.

To start with I used the crazy 8 technique that I had learned in the CSC3731 module to focus on idea generation. This method focuses on generating a lot of ideas quickly, rather than thinking in depth about one idea. Traditionally, the technique involves imagining 8 different designs for each screen. However, I adapted this as I felt completing 8 sketches for the whole system better suited this

project. I took this approach to streamline the process and not come up with unnecessary or repetitive designs while still exploring a variety of ideas.

In these sketches {Appendix B} I explored ideas such as sidebar vs top navigation and card-based vs table-based dashboard layout. I choose to have a top navigation bar as a sidebar would clutter the screen more and there also isn't a lot of pages the user needs to navigate to, although I do feel if I was to scale the program and offer more features this could be a good design change. I also choose a table layout as it felt more professional and allowed me to include more details to help define each plan.

Once I completed this, I looked over all the ideas and took the best aspects of each. I translated them into a Figma design, starting to focus more on a higher fidelity prototype. This meant considering aspects such as colours and spacing in more depth alongside how the program might flow. I focused more on the desktop design as a teacher will spend a lot of their time at their desk planning but ensured to keep in mind responsive behaviour and how it would look on a mobile phone screen. I only designed the key screens that will be a part of my system and make up the user journey. I focused on ensuring these were clear and met the requirements I outlined. I did not include edge cases, error messages, in depth menu navigation.

While designing the high-fidelity designs, I also made sure I was considering HCI principles and Nielsen's usability heuristics [20] to make sure the app is easy to use. For example, I made sure headings and section labels are clear. Navigation is kept consistent and minimal which aims to reduce cognitive load avoiding the use of menus and dropdowns which can cause friction. These support NFR1 and ensure the app is accessible to everyone even with the range of technical abilities within the teaching profession

Landing Page

I designed three different landing pages (Figure 3). I struggled to decide what this screen should look like based purely off wireframes. It didn't take much time to create multiple within Figma as they are not very complex and ultimately, I decided to go with the top left. I initially only had the ones on the bottom. I knew I preferred the left as it seems more professional, but I knew it was lacking something. So, in the final landing page I added more context for the user ensuring the call-to-action button is obvious. I also centralised the logo and welcome message in comparison to the first draft below it which I feel makes a big difference to how good it looks.

This screen ensures the program meets FR1 by allowing users to log in and supports NFR3 by preventing unauthorised access to saved plans.

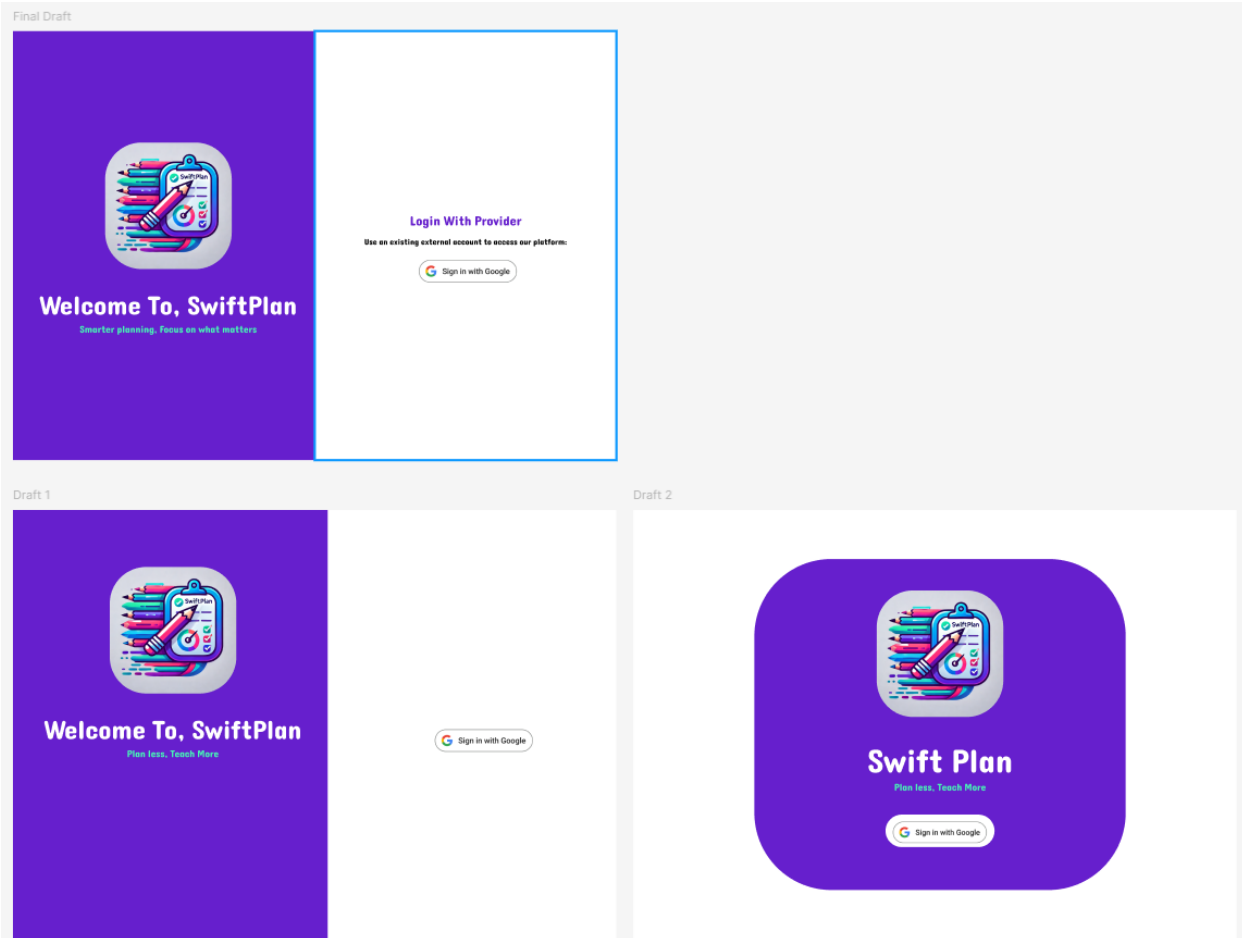


Figure 3: Landing Page Figma Designs

Dashboard Page

In my wireframes I explored using cards and a table-based layout for displaying lesson plans. I decided to go with a table structure as I think it allows for filtering and scanning easier. I think a card layout can sometimes look cluttered and a table layout allows for more information to be portrayed.

This screen (Figure 4) ensures that the application meets FR4 (retrieving saved plans), FR5 (Export to PDF), FR6 (filtering), FR7 (Providing a central repository for saved plans).

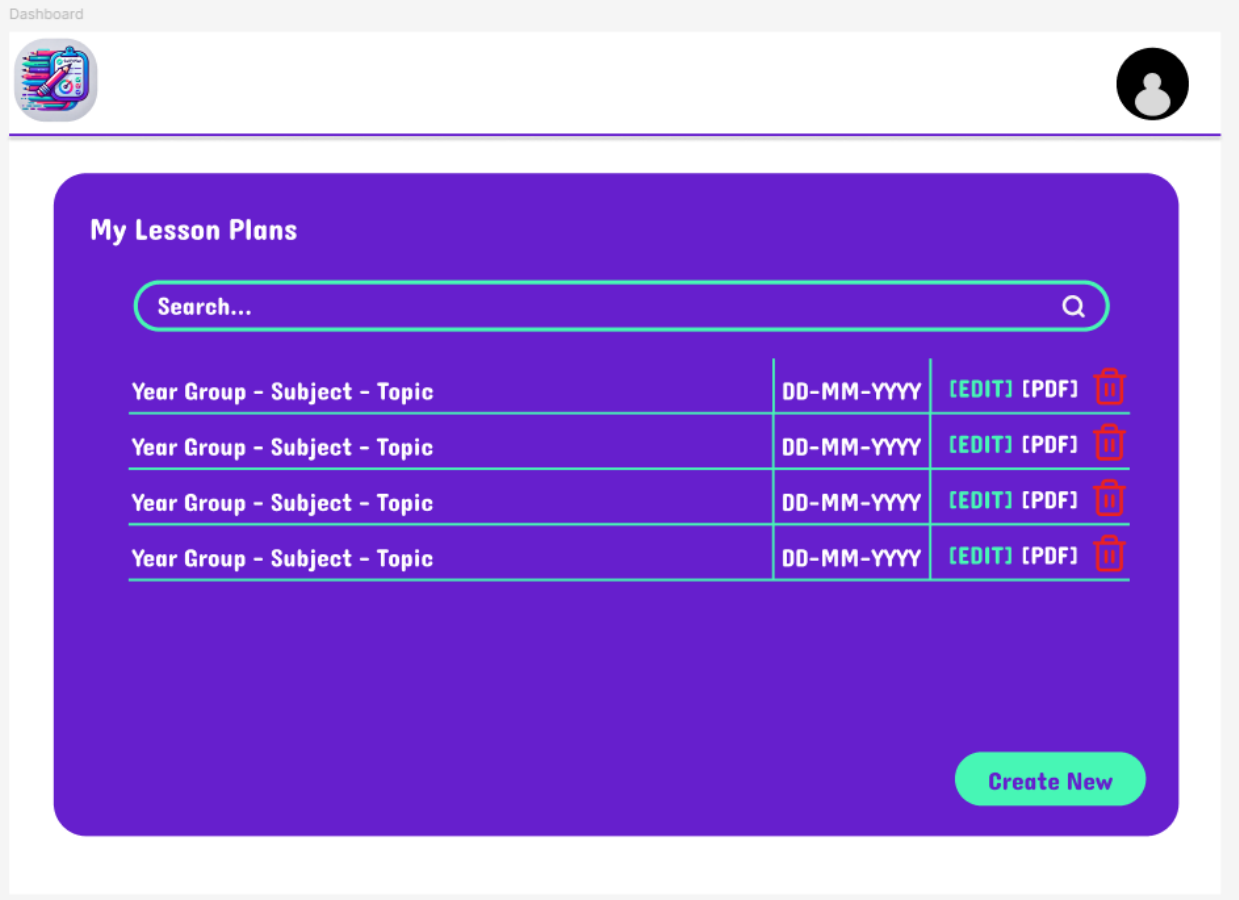


Figure 4: Dashboard Screen Figma Design

Plan Generator

I settled on using a split layout for this screen (Figure 5). On one side the user can enter their parameters which will modify the prompt and tailor the lesson plan. On the other side there is a preview. This supports Nielsen's heuristics and allows system status to be visible. It also creates an efficient workflow that allows for quick adjustments and even better-quality lesson plans. This screen ensures that the program meets FR2 by allowing plans to be generated using AI powered prompts tailored by the teacher.

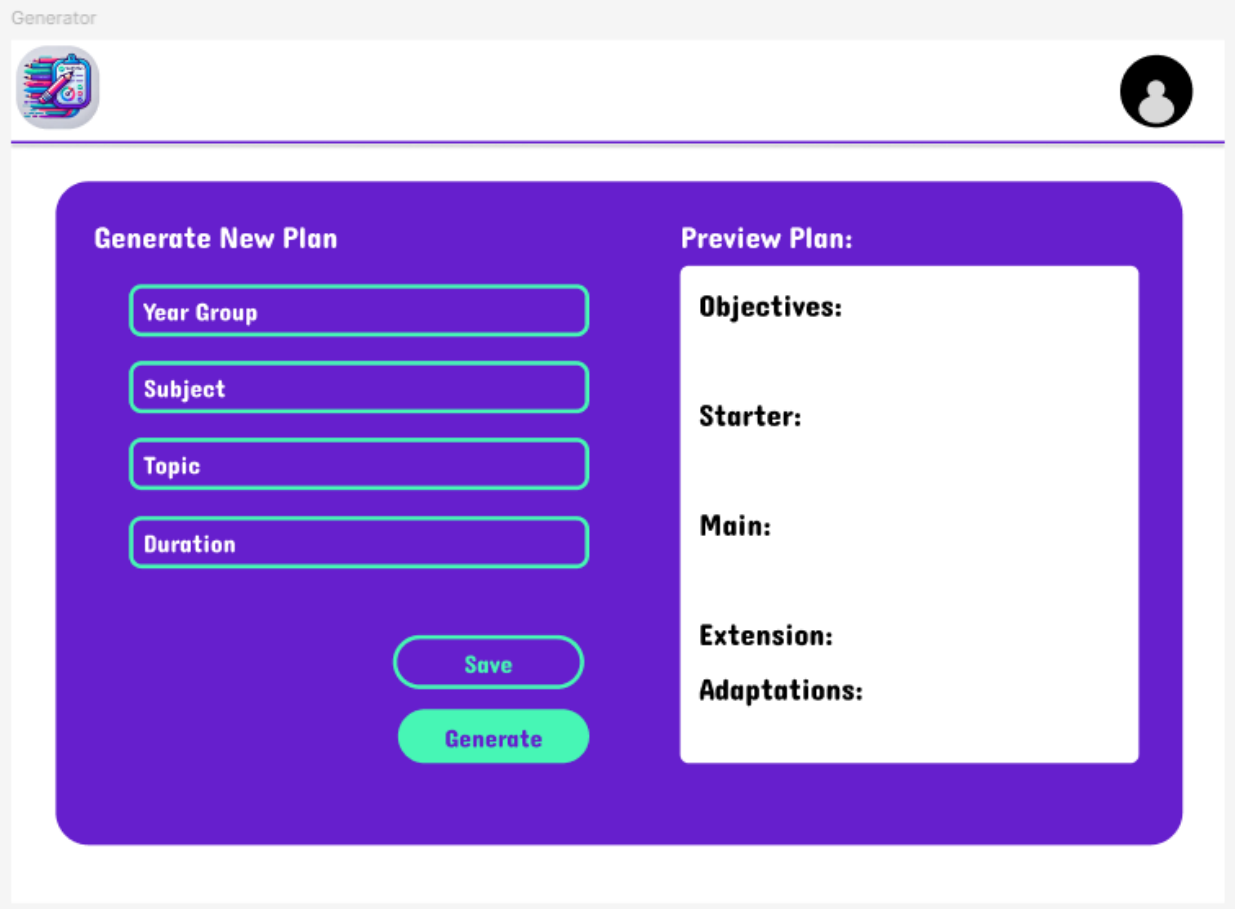


Figure 5: Plan Generator Screen Figma Design

Plan Editor

This screen (Figure 6) builds from the previous “generator” screen but allows full scale editing of the generated plan to allow teacher input. This screen ensures that the app meets FR3 allowing teachers to modify plans where they feel is needed. It also makes sure that the user feels they have freedom and are in control.

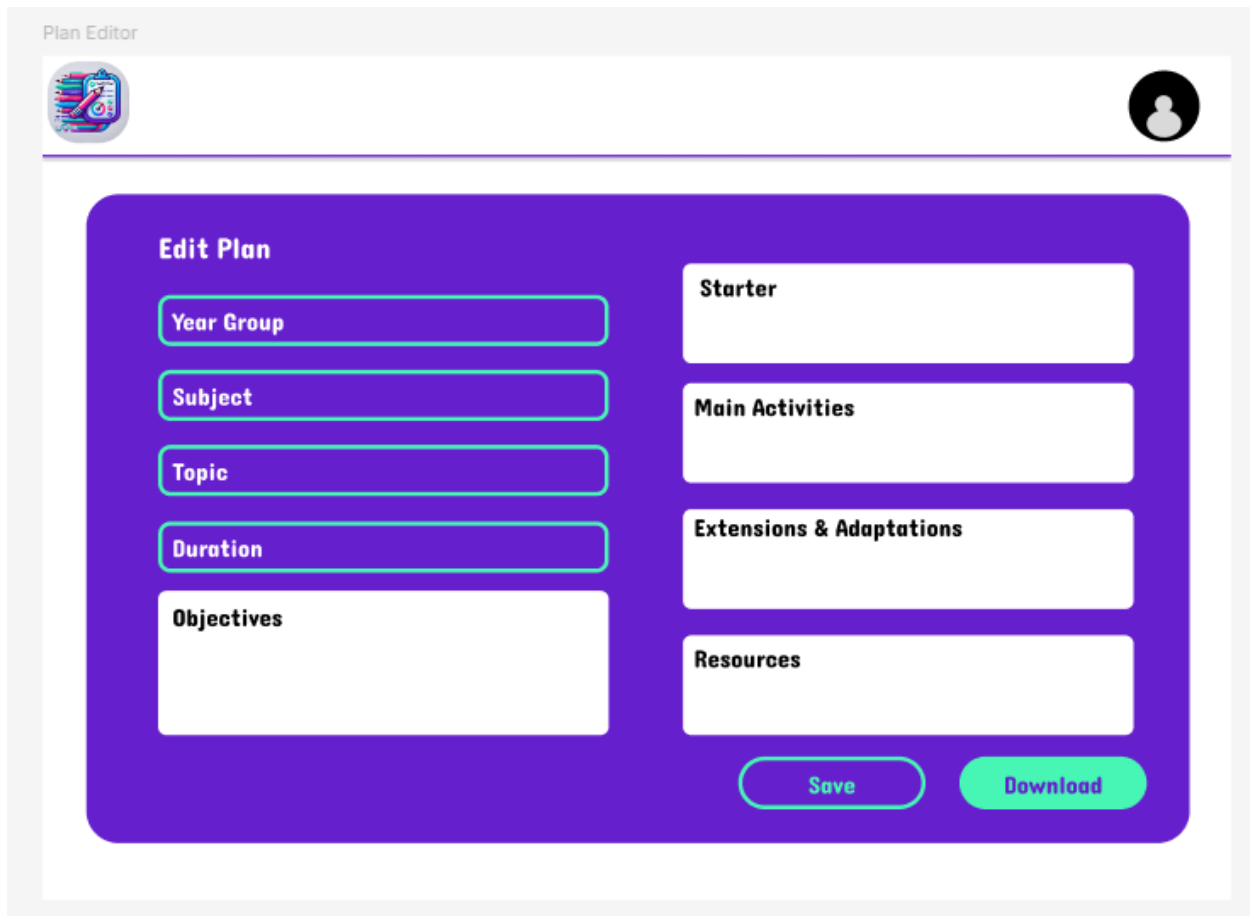


Figure 6: Plan Editor Figma Design

3.7 Integration with OpenAI

The core function of the application (generating a lesson plan) relies on a server-side POST route. Built using the Next.JS app router and using data from a simple form that teachers fill out.

- When the user submits the form, it is validated using Zod. Zod is a TypeScript first tool, but it works well with JavaScript and is easy to implement making it good for my project which is on a tight timeframe. By using Zod I ensured that no null values or edge cases are passed

```

lib > JS lessonPlanSchema.js > ...
1  import { z } from 'zod';
2
3  export const lessonPlanSchema = z.object({
4    title: z.string().min(3, 'Title must be at least 3 characters'),
5    subject: z.string().min(2, 'Please enter a subject'),
6    ageGroup: z.enum(['KS1', 'KS2', 'KS3', 'KS4', 'KS5'], {
7      errorMap: () => ({ message: 'Select a valid age group' }),
8    }),
9    duration: z
10     .string()
11     .min(2, 'Please specify a duration')
12     .refine((val) => /^\\d+\\s*(mins|min|minutes)$/.test(val.trim().toLowerCase()), {
13       message: 'Use format like "45 mins"',
14     }),
15    objective: z.string().min(5, 'Objective must be more descriptive'),
16    style: z.string().optional(),
17    notes: z.string().optional(),
18  });
19

```

Figure 7: Form Validation Schema

into the prompt meaning that the quality of output from the AI is high. It also allows me to easily give user friendly error messages in the case of erroneous values. These help the user fix them quickly and maintains good user experience.

- When validation is completed, the data is passed into the structured template I created in section 3.3. As mentioned, this ensured that output is consistent and makes processing the response easier.
- The request is handled server side and used the API key from the environment variables, so nothing is ever exposed to the client. When the response is received the application uses ## to identify splits in sections and allocate them to the dropdowns in the preview screen.

3.8 Feature Development

I did initially plan to work following the waterfall method but realised this was too rigid and not how I worked best. I followed somewhat of an agile-style workflow. I would pick a task on my kanban board (Appendix A). I would implement it, then test it on the localhost make sure it seemed to work as intended and then pick another card on the board - repeating the cycle.

3.8.1 Landing Page

I started by creating a welcome screen that would allow users to log in using the authentication method outline in section 3.5. Being able to generate/store/edit lesson plans would not be possible without a user that had a valid session, so this was the logical first step and a crucial foundation to build from. The landing page reflects my Figma design and serves its purpose of connecting to Google OAuth. This page also acts as a barrier to entry and users are redirected here if they do not have a valid session token.

3.8.2 Plan Generator

Once the login worked successfully, I moved on to the UI for the lesson plan generator. As discussed in section 3.7 this was done using OpenAI API. This page was split into two sections. The left-hand side contains the form for which teachers enter the details of the plan they need generating. The right side offers a preview of the plan they are generating. It does not re-render on every change to the form as this could mean a lot of API calls, slow down performance and possibly cost a lot. It

could also be annoying for the user seeing the right-hand side keep flashing differently with every word/letter they change. So, the preview is generated when the user clicks the generate button. The preview window makes it obvious to the user that the lesson plan is loading. It pulses purple and shows “loading” to make sure the user is aware of the state of the system which is one of Nielsen’s ten heuristics. It helps to avoid frustration, and this can prevent multiple buttons clicks which would result in many API calls which could bottleneck the network, slow down the performance or have implications on the UI. As mentioned in structuring the prompt I made it, so the response included ## to indicate headers. At first when the preview rendered these were only used as headers (Figure 8). However, upon implementing this it was obvious that the result was long and could be overwhelming to the user.

The screenshot shows a web browser window with the URL 'localhost:3000/LessonPlan'. The page title is 'SwiftPlan'. The main content area is divided into two sections: 'Create a Lesson Plan' and 'Lesson Plan Preview'.

Create a Lesson Plan

Lesson Title:

Subject:

Age Group:

Duration:

Overview:

Additional Parameters (optional):

[Generate Lesson Plan](#)

[Save](#)

Lesson Plan Preview

1. Learning Objectives

By the end of this lesson, students should be able to:

1. Define photosynthesis and explain its significance in the global carbon cycle.
2. Identify and describe the two stages of photosynthesis: light-dependent reactions and light-independent reactions (Calvin cycle).
3. Explain the role of chloroplasts, sunlight, water, and carbon dioxide in photosynthesis.

2. Introduction (5 minutes)

#

Starter Activity

Start the lesson with a short animation or video clip illustrating the photosynthesis process. This will provide a visual context for the topic and engage students' interest. Discuss the video briefly to check prior understanding and to set the stage for new learning.

3. Main Activities (40 minutes)

1. "Direct Instruction (10 minutes)": Present a PowerPoint presentation that covers the basics of photosynthesis, including its importance, the reactants and products, and the two stages.
2. "Interactive Discussion (15 minutes)": Use an interactive whiteboard software to draw and label a chloroplast, discussing the role of each part in photosynthesis. Encourage students to participate in the discussion.

Figure 8: Initial Generator Implementation

I changed the design to follow an accordion style (Figure 9). Each section was collapsible, and this was a lot less overwhelming - the user could choose what section they wanted to view. Another thing I added was a link above the purple card with the main content on that takes the user back to the dashboard. Initially the only route back was by clicking the logo but after using the system myself I realised that although this seemed obvious to me it might not to a teacher who is not as familiar with the interfaces. I also took away some of the whitespace on the edges and think the screen looks better for it. Lastly, improved the form to follow a left to right structure which I learned was good practice in CSC3731 as it follows natural reading direction. I ensured that the main call to action button – “Generate Lesson Plan” – stood out more than the save button as it is the main purpose of the page. The save button also does not appear till the user has generated at least one lesson plan to avoid unnecessary traffic and null values being saved in the database. Once the user has generated and the user chooses to save the plan, I made sure that there was a confirmation message and a redirect back to the dashboard again, so user is confident in the state of the system.

Figure 9: Final Generator Implementation

3.8.3 Dashboard

The dashboard screen aimed to represent a central repository of all lesson plans created by the teacher allowing the application to meet multiple functional requirements in one go: 7 (Dashboard), 6 (Search) , 3 (Edit) , 5 (Download), 4 (Retrieve).

Within the database there is a table for lesson plans that follows this model:

```
model LessonPlan {
  id    String @id @default(cuid())
  title String
  subject String
  yearGroup String
  content String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  userId String
  user   User   @relation(fields: [userId], references: [id])
}
```

The userid acting as a foreign key helps to ensure that plans shown are only the ones by the teacher that is logged in which helps to satisfy NFR3.

Multiple API routes were used to create the dashboard.

- POST /api/generate allows creation of a new plan
- GET /api/lesson returns a list of all plans for a given userId
- PUT /api/lesson/[id] is used to update a specific plan
- DELETE /api/lesson/[id] can remove a plan.
- GET /api/lesson/download/[id] allows file download.

The dashboard UI includes a table that displays all necessary data to help teachers find previously created lesson plans efficiently. There is also a search bar that offers immediate feedback and changes the results based off the search parameters making searching efficient. Every lesson plan has the option to edit delete or download making the user feel fully in control.

3.8.4 Editing Existing Plans

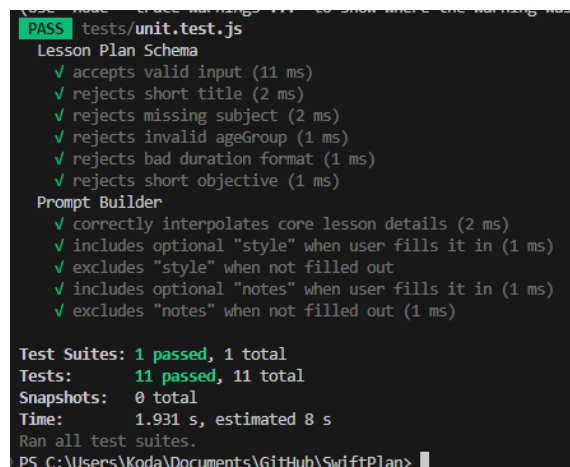
Once I was able to generate and store plans, I could then work on being able to edit them. Allowing the teachers to fine tune the plans to their pupils' needs was important. This kept them in control and ensuring my application was supporting them not replacing them aligning with research from Luckin et al. [21] and ensuring FR3 was met.

I created a dynamic route `/Lesson/Edit/[id]` to enable plans to be fetched by their id and then loaded into an editor that used an imported module, React-Quill. I chose to use this as it did not require much set up, processing of data or configuration. Especially since I was working on this feature in the later stages I was starting to feel time pressure. Quill provided a good enough interface that allowed the edit function to work but did also leave room for improvement in regards to user experience.

3.9 Testing

3.9.1 Unit Testing

To follow software engineering best practice and to verify the reliability of my components I carried out unit testing using jest. I wrote tests for my form validation and prompt builder function. I chose to do these tests as the components being tested directly impact the quality of the data being sent to Open AI. Therefore influencing the quality of the response and consequently quality of my application.



```
PASS tests/unit.test.js
  Lesson Plan Schema
    ✓ accepts valid input (11 ms)
    ✓ rejects short title (2 ms)
    ✓ rejects missing subject (2 ms)
    ✓ rejects invalid ageGroup (1 ms)
    ✓ rejects bad duration format (1 ms)
    ✓ rejects short objective (1 ms)
  Prompt Builder
    ✓ correctly interpolates core lesson details (2 ms)
    ✓ includes optional "style" when user fills it in (1 ms)
    ✓ excludes "style" when not filled out
    ✓ includes optional "notes" when user fills it in (1 ms)
    ✓ excludes "notes" when not filled out (1 ms)

Test Suites: 1 passed, 1 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 1.931 s, estimated 8 s
Ran all test suites.
PS C:\Users\Koda\Documents\GitHub\SwiftPlan>
```

Figure 10: Test Suite

I tested the lessonPlanSchema with both valid and invalid parameters. I tested that:

- Valid input with all required field passes
- Inputs with a short title, missing subject or invalid age failed

- Duration is only accepted if the format is correct
- Objectives must be a minimum of 5 characters to provide some context.

When testing the buildLessonPrompt I was looking to ensure that:

- All required fields were passed in correctly
- Optional fields were included and excluded correctly
- Output has not broken/unnecessary sections

All my tests passed as shown in Figure 10.

3.9.2 Manual Feature Testing

I tested certain features manually on localhost. I used a chrome browser and chrome's devtools, I determined whether a test passed or not by observing UI feedback and API calls/responses. I created a table to keep track of my tests. Screenshots of some interactions are in the Appendix C while tests that have more complex results/effect are explained below.

Table 2 – Manual Testing Results

Test ID	Feature Tested	How it was tested	Expected Result	Result
T1	Login	Clicked “Sign in with Google” button on landing page	Took through google OAuth process and redirected to dashboard.	PASS
T2	Generate Plan	Filled out the form with valid data and clicked generate	Preview displayed on the right side of screen	PASS
T3	Form handles errors smoothly	Deliberately left out duration from the form and tried to submit it.	Error message directing how to fix the error and no API call made.	PASS
T4	Save Plan	Clicked save after generation	Confirmation message, plan shows on dashboard.	PASS
T5	Edit Plan	Followed link to edit page, made a change and clicked save.	Changes to the plan were saved.	PASS
T6	Download Plan	Clicked the download button on dashboard screen	Relevant file is downloaded	PASS
T7	Delete Plan	Clicked the DELETE button on the dashboard screen	File is removed from the database	PASS
T8	Search filters plan correctly	Typed part of a lesson title in the search box	Plans that don't match search parameters aren't shown, ones that do are.	PASS

Test T1 - Login

The application uses Google OAuth to deal with user login. When the app is loaded the user is greeted with the landing page (Figure 11).

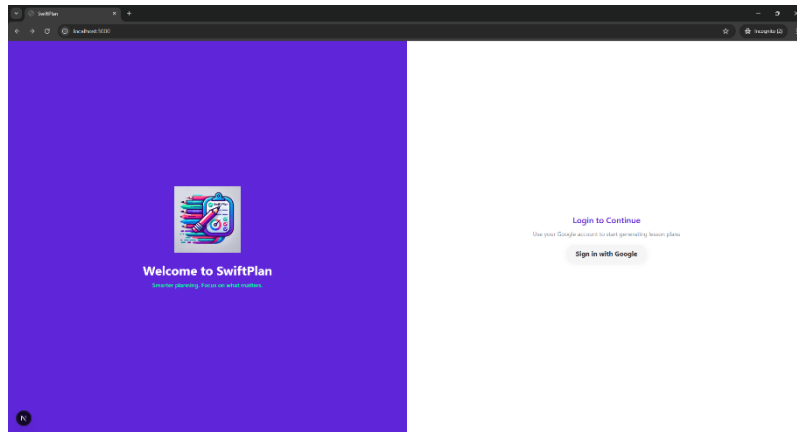


Figure 11: Landing Page

From there they can click “Sign in with Google” that re-directs them to the Google login flow which they would be familiar with (Figure 12).

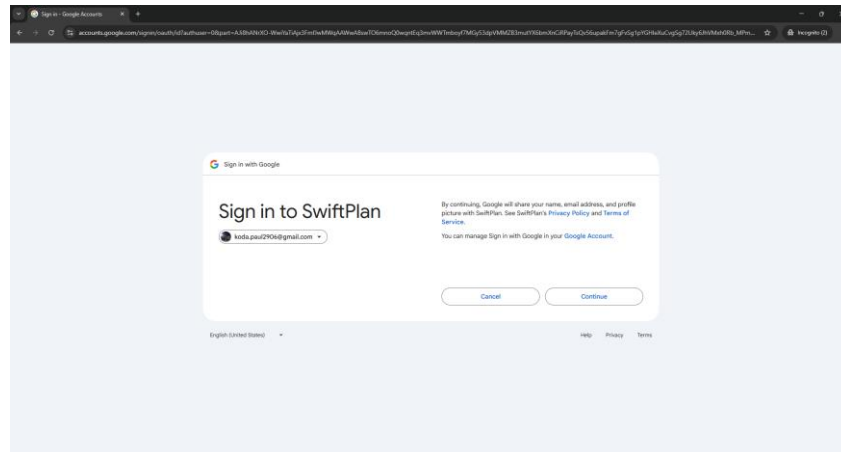


Figure 12: Google OAuth Login

Once this is completed and the application process the session authentication, the user is returned to the dashboard screen (Figure 13). The profile picture of the logged in user appears top right and acts as the menu button to access the logout button.

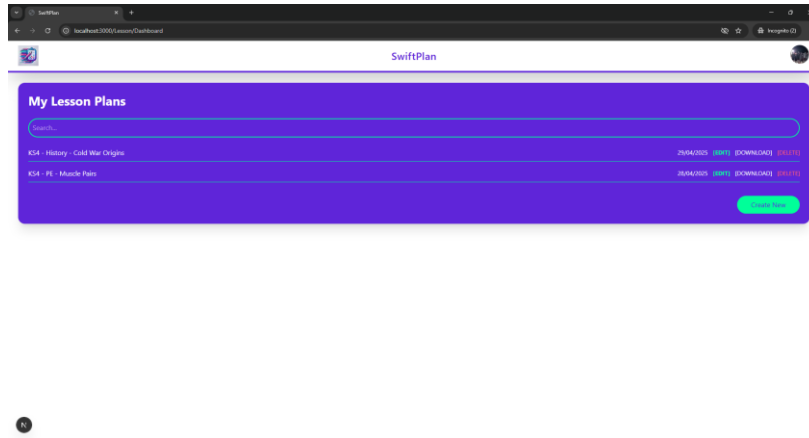


Figure 13: Dashboard Screen After Successful Authentication

Test T2 – Generate

From the dashboard screen (Figure 13) the user can click the “Create new” button which takes them to the generator screen. Where they can enter parameters into the form and click “Generate Lesson Plan” (Figure 14).

Figure 14: Generator form screen

While the plan loads the form ensures the user is aware that the process is taking place to avoid frustration. (Figure 15) Then displays a preview of the lesson plan (Figure 16) so that the teacher can make changes and regenerate if needed before saving it. In Figure 16 the successful API call can be seen as a part of the network log. This is an example of how I assessed the speed of the response. I did this three times and recorded the results. On average the response time was around 30 seconds. Although this means the test did pass it did take longer than I would like.

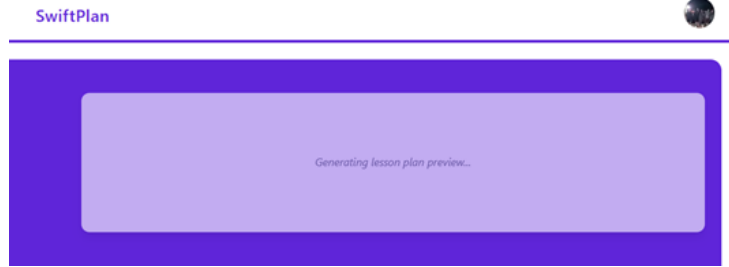


Figure 15: Loading State shown while generating plan

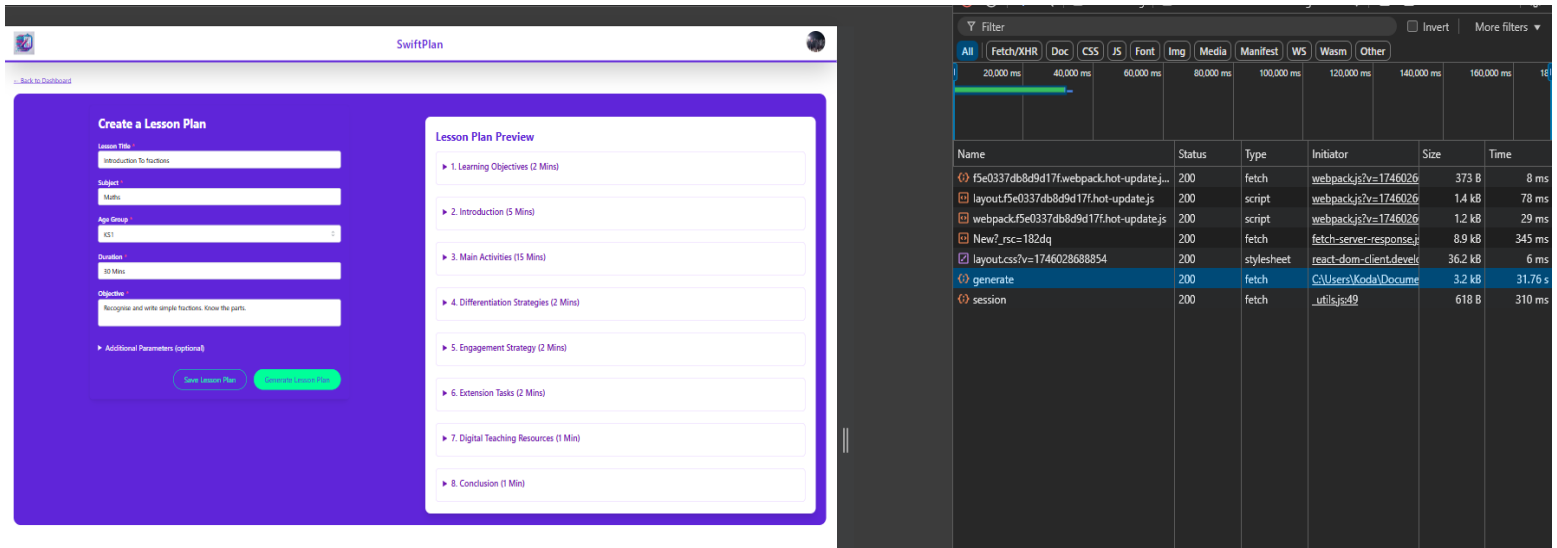


Figure 16: Complete Lesson Plan Preview with Network

3.10 Summary of Development and Tech Stack

Although initially planning to carry out the waterfall method this I changed to somewhat following agile principles. Working iteratively using my Kanban board and Gantt Chart to track progress, I focused on the backend first. This involved building the database schema and API routes to allow lesson plan generation, storage, and retrieval. Once I had this implemented, I had a better understanding of how the frontend would need to be developed for users to interact smoothly with the backend.

The first screen I built was the landing page and connected to NextAuth as I needed the user to be able to log in to have an id and session that I could link to the generated plans. Then I built the generator page as I needed to be able to generate lesson plans before I could start on the other features – saving, editing, searching, downloading, deleting. Edit was next challenge I took on and this was simultaneous with saving as they went hand in hand. I choose to do this second as I was integrating with React Quill which I knew would take more time than just writing API routes for deleting and downloading plans.

I used GitHub for version control and it also helped tracking my progress. My commit history shows how I worked incrementally. I also ensured to give my commits sensible names, and I tried to remember to start all my commits with the main action of the change such as, “ADD” or “REFACTOR”.

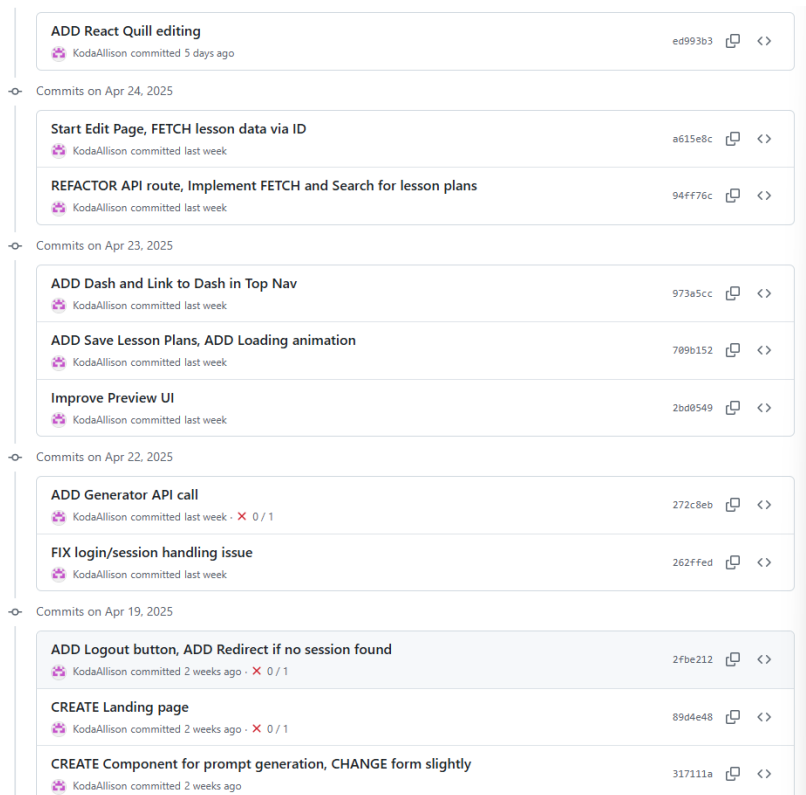


Figure 17: Commit History Snippet

This also meant that I could rollback to previous working versions when I got stuck especially because I had so many components linking together, I occasionally did get confused and decided it was more efficient to rollback and redevelop then try to find what broke the project myself.

3.10.1 Coding Standards

I tried to follow best practices where possible to ensure that my code was readable, maintainable and therefore scalable to make it easy if myself or others were to complete work on it in the future. For the most part I did a good job of this, I made use of appropriate comments, used camel casing for my variables and functions then used pascal casing for my react components this was to distinguish them within files. As mentioned, I kept track of progress using git, giving appropriate names to commits and attempting to follow a consistent format. However, I could have made use of branches more effectively to separate finished code from features that were still being developed, which is considered good practice in industry. I also did not fully implement continuous deployment, which is something I would improve on in future projects

I made use of ESLint with Prettier (an IDE extension) to ensure my code was consistent. This meant that I had no issues with as spacing, indents, missing semicolons or unused imports. Helping me to follow best practices and have readable, maintainable code.

My folder structure (Appendix E) made navigating the codebase as easy as possible. I had dedicated folders to group files together logically:

- app/api – stored backend API routes
- Components – stored reusable UI components, improved modularity

- Prisma – stored the schema for my database along with any migrations when syncing with the live database.
- Tests – stored unit tests for logic and form validation.
- /app/Lesson – stored all frontend pages such as /Lesson/Dashboard or /Lesson/Edit. The layout folder for these pages prevented unauthenticated users from accessing.

3.10.2 Technologies Used

The full list of technologies used in the application:

- **Next.js** – I used Next because it can handle the full stack – the front and back end in one framework. It has built in API routes which allowed me to create endpoints in the same codebase. This meant I did not need a separate backend server, saving setup time, avoiding complexities in not having to integrate frontend with backend and potential costs. I made use of API routes for database operations, authentication and interacting with Open AI without exposing sensitive info to the front end. Furthermore, Next allows for server and client components which improves performance. I used server components when I needed to fetch data and client components for interactive elements of the application. This meant loading times for pages were quick as the server only renders necessary JavaScript for each page and some content is loaded by the server before it is sent to the browser.
- **Tailwind CSS** – Made styling easier by providing predefined classes that can be applied to elements. This helped to ensure consistency as the classes were pre-defined and sped up the process of styling my application because I did not have to write a custom CSS file.
- **Prisma** – Allowed me to interact with the database from inside the application. I used it to define my schema models and to run migrations with the database so that it was synced with my schema. Also meant I could safely query the database without writing raw SQL. I could use simpler commands such as “prisma.lessonPlan.findMany({ where: { userId: session.user.id } })” which would fetch only the plans that belonged to the currently logged in user. This meant code is more readable, easier to debug and safer as there is reduced risk of SQL injection.
- **PostgreSQL** – A relational database hosted on Neon and connected via Prisma Allowed me to store users, their lesson plans and session data. The relational model suited this project as it allowed me to manage the teacher and plan relationship easily.
- **NextAuth.js** – Authentication using Google OAuth meant I did not have to write my own logic for user authentication/management also easier for users as they do not have to remember another user/password they can use a pre-existing account. It helped to increase security as I could use session management methods provided by nextauth to protect endpoints and ensure only logged in users could access their lesson plans.
- **OpenAI API** – I used this for generating structured lesson plans based on teacher input. Structuring a prompt based off my teacher survey and researching prompt engineering techniques. Users passed specific details into the prompt via a form, so the plan generated suited their needs.
- **React Quill** – Module imported into my codebase that allowed for quick set up of a text editor for the generated plans. Although it has its limitations, it still provided edit functionality.

- **GitHub** – I used this for version control. Helped me track changes, allowing me to go back to the last version that worked where needed and debug what went wrong.
- **Jest** – I used this for completing unit testing. I wrote tests that verified my form validation logic in the lesson plan generator form worked as expected. I also included tests that made sure only valid data was passed into the prompt. These ensured my application behaved as expected and was reliable.

4 Results and Evaluation

4.1 Functional Requirements Review

Table 3 – Functional Requirements Evaluation

Requirement	Description	Met?
FR1	User can log in using google OAuth	Yes
FR2	System must generate lesson plans based off of AI prompts	Yes
FR3	System must allow for teachers to edit lesson plans	Yes
FR4	System must be able to store and retrieve past lesson plans	Yes
FR5	System must allow user to be able to export lesson plans as PDF	Partial
FR6	System must allow users to search stored lesson plans	Yes
FR7	System must have a dashboard for managing lesson plans	Yes

I tested these manually as seen in section 3.9.2. FR5 is the only one of the functional requirements that was not fully met but, it was still partially completed. Originally planning for lessons to be downloaded as a PDF this changed during development. Plans can still be downloaded but they are as a .txt file instead. This decision was made as it was easier to implement and the file that was downloadable still gave a good, structured outline of the lesson plan (Figure 18).

```
## 1. Learning Objectives (5 mins)
- Students will be able to define and distinguish between agonist and antagonist muscles.
- Students will be able to identify examples of muscle pairs in the human body.
- Students will demonstrate understanding by explaining how muscle pairs work together during physical activity.

## 2. Introduction (10 mins)
Begin with a short interactive quiz to assess prior knowledge on muscles, using an online tool like Kahoot. Questions should cover basic muscle knowledge. This will not only capture students' attention but also provide a baseline for the day's lesson.

## 3. Main Activities (30 mins)
- **Activity 1 (15 mins):** Use an interactive digital whiteboard to present a short lecture on muscle pairs, focusing on agonist and antagonist muscles. Use digital diagrams to illustrate how these muscle pairs work together when we move.
- **Activity 2 (15 mins):** Break students into small groups within a breakout room on the video conferencing tool. Assign each group different muscle pairs to research and prepare a short presentation. Students will use online resources to find information and create a digital slide presentation to share with the class.

## 4. Differentiation Strategies (included within activities)
- **SEND Support:** Provide audio-visual resources for students with reading difficulties. For those with attention issues, break tasks into manageable chunks and provide clear, step-by-step instructions.
- **Different Learning Styles:** Incorporate activities that cater to visual (diagrams), auditory (lecture), and kinesthetic (interactive quiz) learners.

## 5. Engagement Strategy (included within activities)
The group work and presentation activity require students to conduct their own research and share their findings, promoting peer learning and active engagement. Using breakout rooms for group activities encourages interaction and collaboration.

## 6. Extension Tasks (10 mins)
For advanced learners or early finishers, assign a task to research and present on the role of muscle pairs in a specific sport or physical activity. They should explain how understanding this concept can improve performance in the selected sport.

## 7. Digital Teaching Resources
- **Quiz:** Create a digital quiz using Kahoot or Google Forms, with questions covering basic muscle knowledge.
- **Diagrams:** Use digital diagrams available from free online resources like Pixabay or Unsplash to illustrate muscle pairs.
- **Research:** Direct students to reputable online resources like BBC Bitesize or Khan Academy for their group research activity.

## 8. Conclusion (5 mins)
```

Figure 18: Downloaded lesson plan output

Using .txt format required no extra dependencies and did not need the content to be formatted further than what was already done to provide a good UI in the application – unlike if I was to implement downloading of a PDF. Therefore, this decision saved time in the development process but still allowed for offline access and sharing of the plan just not how I originally planned. The prompt used to generate lesson plans was directly informed by teacher feedback (Section 3.2). Question 5 of the teacher survey asked, “What are three things a lesson plan must include to be useful?” The most common answers were clear learning objectives (100%), precise activity timings (over 80%), and effective differentiation strategies (over 50%). These elements were explicitly included in the final prompt structure and can be seen in the lesson preview (Figure 19) ensuring that each generated lesson followed a format that matched expectations.

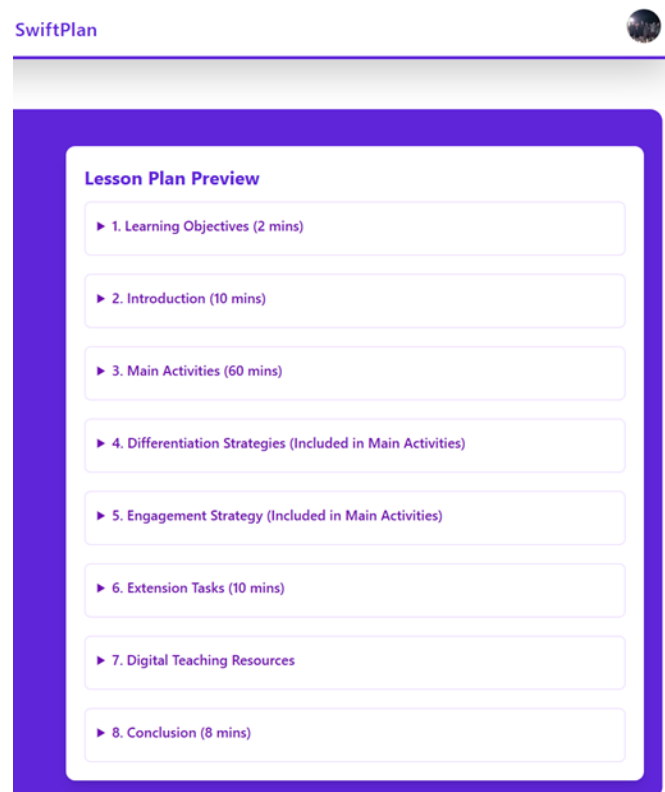


Figure 19: Lesson Plan Preview

Other than the file format being different to originally planned, all functional requirements were met meaning my application satisfied my goals, a user can log in, generate a plan, save it and then export it if required.

4.2 Non-Functional Requirements Review

Table 4 – Non-Functional Requirements Evaluation

Requirement	Description	Met?
NFR1	System must follow Nielsens Heuristics	Yes
NFR2	AI generated responses must have fast turnaround time	Partial
NFR3	System must be secure with no unauthorised access to lesson plans	Yes

NFR1 – I carried out an informative think aloud evaluation and kept the heuristics in mind throughout the whole design and development process. In particular: consistency, minimalism, visibility of system status and error recovery. I think it is obvious in the final product that I did so. Also, other common HCI principles such as left to right forms were kept in mind and this resulted in a user-friendly UI that took minimal cognitive load to use and learn.

NFR2 – I used console logs and Google DevTools to assess how long the open ai ‘generate’ request took (See Appendix G). I trialled it multiple times with the exact same parameters and making sure to clear the cache beforehand to keep as all variables the same. On average the response time was 26.64 seconds. I did not set a goal of an exact time that I wanted the turnaround time to be, but I do think this is slower than I had hoped. However, it is still quicker than if a teacher was to spend multiple hours a week planning a lesson. Therefore, there is room for improvement as it still meets the goal of quickly generating lesson plans.

NFR3

Functionality of the app is protected, and the user is redirected to the landing page if they are not logged in and have a valid session. Each lesson plan is stored with a userID that acts a foreign key. This means when plans are fetched the query uses this ID to ensure the user only accesses their own plans. Furthermore, a .env file is used to store the OpenAI API key along with credentials for Google OAuth.

4.3 Performance

I measured performance during manual testing by assessing how long it took to come up with a lesson plan. I sent multiple separate generation requests that all contained the same parameters, also ensuring to clear cache beforehand. The results averaged 26.64 seconds (Appendix G). This is slower than I initially hoped to have in my implementation. However, it is still significantly quicker than spending multiple hours a week planning lessons using traditional methods.

This slower response offers room for improvement but does not take away from the fact the program is still functional.

4.4 Usability

4.4.1 Lesson Plan Usability and Customisation

Even though the application can generate lesson plans relatively quickly, the goal of saving teachers time will not be met if those plans are not usable. As shown in section 3.2 the survey given to teachers identified that a plan should contain: clear learning objectives, precise activity timings and effective differentiation strategies.

Upon review of the lesson plans generated by the application (Appendix D) I have met these expectations. Each plan contains well labelled sections, consistent formatting, clear timings for each task/section and provides strategies for differentiation and engagement.

To enhance usability further, I made sure it was possible for teachers to edit plans so they could tweak them and make necessary adjustments. This is also in response to the teacher survey, when asked if they were open to AI-generated plans one participant stated “Yes, but I’d want to adapt the AI generated content”.

These plans were not shown to teachers, but the structure and content do reflect what I learned in the survey. The fact it aligns with what was said in the survey shows the plans can be useful in a classroom setting. However, to fully assess the usability, they should be used and evaluated in classrooms by real teachers.

4.4.2 Application Usability

To evaluate the usability of my application, I conducted a heuristic evaluation based on Jakob Nielsen’s 10 usability Heuristics [21] by completing an informal, simplified think aloud protocol because of time constraints. Ideally this would have been done with multiple participants that were a part of the target audience but because this is an individually ran project, I was not able to do this. So, my evaluation was completed with a fellow computer science student who I asked to try imagining themselves as a teacher. In addition to this, I also analysed the system myself. Evaluating how well I thought I met each heuristic from a developer’s perspective.

I gave the user a realistic scenario, asking them to complete six tasks that reflected a typical teacher workflow: logging in, generating a plan, previewing the result, navigating back to the dashboard, editing a saved plan, and downloading a plan. During the session, I downloaded a template to take notes (Appendix F) that provided guidance on carrying out the evaluation. I observed their behaviour and verbal feedback as they completed the tasks I assigned (Table 5). I asked to follow up questions to clarify points of confusion or positive feedback.

Table 5 - Think Aloud Task Observations

Task	Observation	Relevant heuristic
Logging in	Login was obvious. Using google felt familiar and I like how I do not have to create and remember another password.	Visibility Of System State, Recognition over Recall
Generator form	I like it all in one page and the form is clearly labelled. Error handling is nice too I deliberately entered something wrong, and it prompted me in a nicely to fix it"	Aesthetic and Minimalist Design & Error Prevention
Plan generation	It felt slow but I liked how the preview box showed me it was loading so I knew that something was happening.	Visibility of System Status
Dashboard	It is easy to search plans, I am not overwhelmed by too much info - just their title. Buttons to manage plans are obvious, I like how delete is red.	Flexibility and efficiency of use, error prevention, Aesthetic and Minimalist Design
Edit screen	This screen feels clunky. I find it hard to locate which part of the plan I want to edit.	User Control and Freedom & Aesthetic and Minimalist Design
Download	This was easy to complete the button to download plans was obvious, I like how the name is relevant, and the formatting of the file is nice	Flexibility and Efficiency of use, Recognition over Recall

Due to time constraints, I did not fully implement any changes to the system because of this think aloud session. However, I did use it to help write my future work section (5.3).

Below is a summary combining the think aloud and my own evaluation of what was done well in relation to all ten of Nielsen's heuristics.

Visibility of system state:

- Loading placeholders can be seen in key areas such as when fetching plans for the dashboard (Figure 20) and generating a plan (Figure 15).
- Confirmation messages are shown when saving and editing plans which give user confidence in the system alongside error messages that offer clear and obvious fixes to allow continued use of the system.

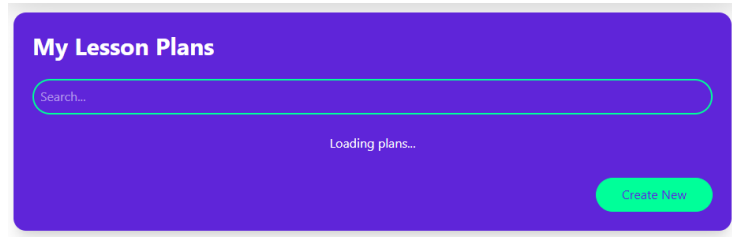


Figure 20: Loading State shown on Dashboard

Match between system and real world

- Use terminology that teacher's use so the application feels natural. Plans have headings such as: "Learning Objectives", "Activities", "Starter" and the form uses fields such as "Subject", "Year Group" and "Objectives".

User Control and Freedom

- The dashboard acts as a central hub where teachers can review all their saved plans. Each plan they create is shown here. Buttons are clearly labelled allowing them to easily edit, delete and download plans without having to use menus. The ability to edit plan makes sure the user still feels in control of what they are teaching, they are not locked down to what the program generates abiding by what I learned from Luckin et al[21].

Consistency

- Layout and colour scheme are all kept the same throughout the system. All action buttons such as "Create New" and "Generate Lesson Plan" are styled the same and always are positioned on the right with secondary actions being an outlined version of primary style. They are always placed to the left or below.

Error Prevention

- Form validation using Zod ensures that data entered is correct. Meaning it cannot provoke an error in the API call/database storage and that the lesson plan returned will be suitable for the teacher.

Recognition over recall

- Buttons are clearly labelled with their purpose and do not use discrete icons that would require time to learn the meaning of. The search system uses plan titles, so the user does not have to know the plan id.

Flexibility and efficiency

- The dashboard search feature is responsive and provides live feedback as the user types. This is more efficient than if the user had to submit a search form possibly as part of a different screen or prior to the results being loaded which would also make the experience feel clunky.

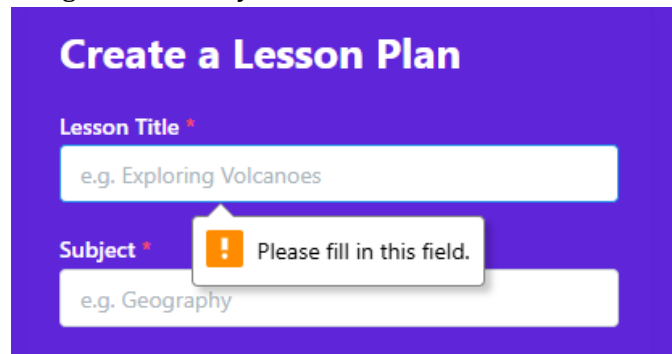
Aesthetic and minimalist design

- I used whitespace to avoid cluttering the screen and only used the title of lesson plans for browsing - hiding the content to not overwhelm the user and keep it minimal. I used bold contrasting colours for buttons so they stood out. Also, the form only asks for necessary inputs to generate a plan and hides additional parameters behind a dropdown so they can still be used if the user wants to.

-

Helps users recognise, diagnose and recover from errors

- When the form is validated if the data is incorrect users are shown a message that clearly describes what is wrong and how they can fix it.



The screenshot shows a form titled "Create a Lesson Plan" on a purple background. It has two input fields: "Lesson Title *" with the placeholder text "e.g. Exploring Volcanoes" and "Subject *" with the placeholder text "e.g. Geography". The "Subject" field is empty, and a white tooltip with an orange exclamation mark icon and the text "Please fill in this field." is pointing to it.

Figure 21: Error Correction Message for missing field



The screenshot shows a form titled "Duration *" on a purple background. The input field contains the text "forty five" in red. Below the field, there is a red error message that says "Use format like '45 mins'".

Figure 22: Error Correction Message for invalid field

Help and documentation

- Ensured the design was as self-explanatory and predictable as possible as no full help page or tutorial was created. To do this, I ensured buttons were clearly labelled with their purpose, avoided using ambiguous icons and kept navigation simple. I also minimised the number of steps it took to complete a task. Such as, having the user fill out the full form at once rather than one field, then clicking next to advance to the next field and so on.

4.5 Limitations

The system can do what I set out to but I am unhappy with a few finer details that would result in a more polished final product. As previously mentioned, I was not happy with the speed at which the app returns a lesson plan after clicking generate.

I was also not happy with a few things on the edit page. The save button was always shown, I had planned to have it show as deactivated and dimmed when no changes had made to improve visibility of system status however, I ran into issues implementing this. The way Next.JS renders the page made it awkward to compare original content with edited version. The system would end up using a null value as the "original content" as the data was not rendered yet. So even though nothing changed it thought there was a change due to comparing what was in the editor against a null value. I decided just to ignore this and focus on core functionality. Another thing on this screen I was not happy with was the actual text editor provided by react quill. The editor loaded a full block content of the plan that appeared in one large unstructured paragraph (Figure 23). Although this is functional it is not a good user experience and means making small tweaks is quite difficult.

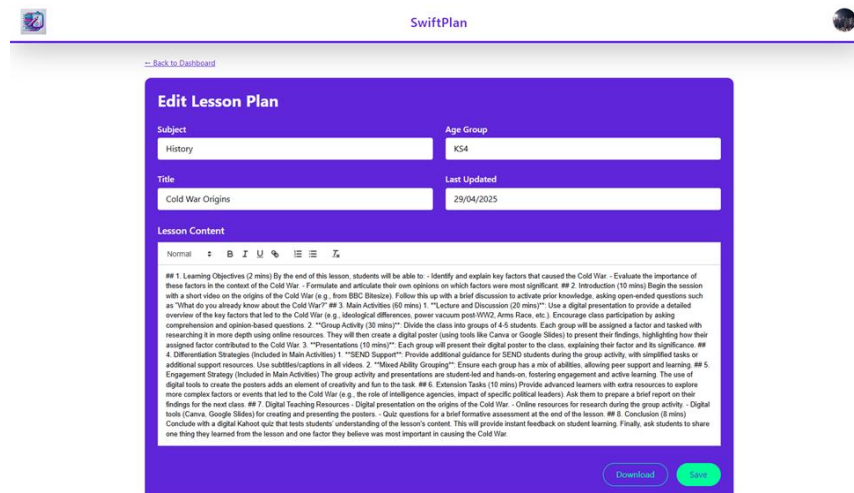


Figure 23: Edit Page



Figure 24: Download Confirmation

When using my personal browser (Figure 24) I receive confirmation of a download, there is no actual confirmation from the application. This means there is a possibility depending on individual browser setup that the user does not know if their download has worked which could lead to some confusion and frustration.

Even though I made sure I could use the application during development I would have also liked to conduct more in depth and structured formal testing. I planned on having the product finished earlier and to carry out user testing with real teachers. This would have included an improved version of think-aloud testing with actual users and formally assessing the impact the application could have in a classroom environment.

Even though I did conduct some performance tests this was limited, and I would have liked to assess how the database would cope if the app was scaled to have more users and how the application performed on different devices and browsers.

I kept in mind good security practices during development but again I would have liked to of conducted further tests to assess whether these worked. If I had more time I would have attempted to bypass the authentication I implemented, simulate attempts at unauthorised access and testing whether the URLs and API endpoints were protected like I intended.

Although these limitations do not hinder the application carrying out the main requirements, they do take away a level of professionalism and polish.

4.6 Overall Effectiveness

Overall, throughout the course of this project I was able to complete a streamlined process of the software development cycle. This included gathering requirements, designing the interface and system architecture, developing features and to an extent testing the application.

My kanban board (Appendix A) helped me to stay on track and prioritise what tasks I needed to do next in order to develop the core functionality, This meant I did not get too caught up in the finer details of some features that wouldn't add much to the end product..

The application for the most part met its requirements although there is room for improvement.

5 Conclusion

5.1 Summary

The aim of this project was to reduce the pressure on teachers by helping them spend less time on lesson planning, without sacrificing the structure or quality of the plans. I aimed to do that by building a tool that could generate digital lesson plans using generative AI and allow teachers to save, edit and reuse said plans easily. I believe the final system achieves that aim and while there's still plenty that could be improved, fundamentally it achieved my goal.

All of the main features - generating, saving, editing, searching, downloading, and deleting lesson plans - were fully implemented and tested manually. I didn't run a full usability study, but I did carry out some scaled down testing and constantly used the app myself while building it to ensure it worked as I expected.

5.2 Evaluation of Aims and Objectives

I met my aim of developing a web app to generate lesson plans which hypothetically will reduce the workload of teachers. The lesson plans include enough content to maintain a good standard of teaching.

Objectives:

1. **Research Prompt Engineering:** I met this objective by using both academic resource and online blogs to build my knowledge of modern prompt engineering techniques. I developed a good understanding of what makes a good prompt – such as role assignment, structure, output scaffolding - this enabled me to have a prompt capable of generating good plans that are effective and usable.
2. **Conduct a teacher survey:** I successfully carried out a survey that solidified the need for my solution, reinforced the claims I made about workload and provided useful insight into how teachers currently plan lessons.
3. **Develop structured AI prompts:** I successfully combined insight from the survey with what I learnt about prompt engineering techniques. I came up with a prompt that was capable of generating a lesson plan tailored to the what the user asked for in the application. Plans clearly outlined timings, tasks, and other aspects. Clear headings made them easy to follow. The prompt can be seen in Section 3.3 and example plans that were downloaded can be found in Appendix D.

4. **Create a requirements table:** This objective was met, and the requirements table gave me clear objectives which helped outline the steps I needed to take to meet my aims and objectives. It helped structure my development and reassured me I was making progress in times of doubt and gave me motivation when I could tick off meeting one.
5. **Create a high-fidelity prototype of the User Interface:** I met this objective by creating wireframes using a “crazy eights” inspired approach. This allowed me to come up with a lot of ideas quickly, comparing what would work best for this system. Then I used this to come up with designs in Figma that considered colour schemes and various HCI principles such as consistency, visibility, user control. I did not fully design things such as state interactions for buttons or error handling but did enough so that it was a good reference while developing the app. This meant I was not also thinking about the design aspects and making design related decisions which would have added stress during development and probably took longer.
6. **Implement backend API:** I successfully created API endpoints using the NextJS app router. This meant it was easy to interact with OpenAI and my own database, allowing me to add basic CRUD functionality to the applications. I implemented standard HTTP methods such as GET, PUT and DELETE, following RESTful principles to make sure that the application was robust and followed best practices.
7. **Develop the web application:** I was successful in developing a fully functional web application that used a variety of modern technologies. The app supported CRUD operations: teachers are able to generate, view, edit, update, delete and download lesson plans.
8. **Integrate working Authentication** – To successfully meet this objective I used NextAuth to implement user registration, login and session handling quickly and with minimal complexity. This meant my application was secure whilst saving time not having to write too much bespoke code. It also allowed users to log in to the application using an account that already exists which makes this experience quicker for them too. This allowed me to link lesson plans directly to a user letting me ensure that they were only accessible by the creator.
9. **Evaluate application:** This objective was only partially met. I carried out manual testing to ensure that the application met my expectations when generating plans. This included ensuring plans had the key elements – timings, clear section headers etc – and that CRUD actions were supported. I also implemented a small amount of unit tests, but their coverage wasn’t quite what I would have liked to achieve. Also, although I personally evaluated the application and how well it met Nielsen’s heuristics I did not conduct a full think aloud walkthrough. I also intended on having usability validated by teachers.
10. **Evaluate effectiveness:** This objective was not fully met. A basic comparison can be made between time spent planning lessons manually and time taken by my application, but I did not gather any formal quantitative data. The app currently averages around 25-30 seconds to generate a lesson plan. It is not unreasonable to suggest that a teacher plans around 20 – 25 lessons a week and therefore it would take 10-15 minutes to produce a full week of lesson plans. Some survey participants indicated they spend up to 4-8 hours planning lessons, and the Ofsted report [1] stated teachers work up to 50 hours a week with a lot of

that on non-teaching tasks. This suggests that this application can save teachers a significant amount of time. However, without formal data this cannot be confirmed.

5.3 Future Work and Expansion

The product I have developed offers a good foundation and achieved the core functionality I planned for but also has a lot of room for improvement. This includes performance, usability, testing and even possible expansions into other features. Using the think aloud evaluation, unit testing and my own manual test here is an outline of what can be done to improve the current version:

5.3.1. Fundamental Improvements

Improve Generator Performance

The current results are acceptable but can be improved. I would aim for at least below a 10 second generation time and possibly even lower as people are becoming more expectant of almost instant loading times. There are a few methods I have thought of that I could do to improve the speed:

- Allow teachers to select what sections they want to return
- Use a different gpt model or at least offer the choice to and explain the trade-off between using different models.
- Rather than rendering the whole plan at once. Look at rendering each section incrementally as it is processed to give an illusion of less time spent on the generation of a plan.
- Look at techniques to reduce prompt length or complexity whilst maintaining the quality of a plan.

Improve Editing Experience

The current method for editing is messy and each part of the plan is in one singular, unstructured block. In the future, this can be improved to look more like the preview screen. The screen could have collapsible sections that allow the user to pick what section they want to edit and hide the rest. Furthermore, an autosave feature of the new draft could be implemented to avoid the user losing changes if they accidentally close the browser or go to a different page.

Formal Testing

Although I used the app as the developer and it worked as intended, more extensive, formal testing would improve my confidence in the usability and security of the platform by showing actual evidence. To gain further insight into the usability, testing with real teachers should be carried out. This could include an improved version of the think-aloud evaluation using the target audience to see how it meets expectations and fits into the classroom environment. Also, formal security testing should be carried out to ensure the application is robust, including:

- Trying to bypass authentication and attempt access to protected routes.
- Test unauthorised access to API endpoints
- Validating that users can't somehow fetch other users' plans

5.3.2 Future Expansion

Once the current implementation is refined and the lesson planning features are fully completed there is a lot of potential features that could be added to help reduce teacher workload further. One potential feature is an AI quiz generator that could come up with formative assessment based on the lesson plans.

Another idea is a resource generator that could help with producing PowerPoint slides or worksheets.

Alternatively, another possible feature is AI assisted feedback. The system would take students work and provide suggestions for marks and feedback with teachers still having the final say if needed but this does have the potential to make the marking process easier.

5.4 Final Reflection

From the start of the project, I dealt with personal issues and challenges. I regularly felt behind in terms of progress in comparison to peers. Despite this, I am proud of the way I pushed through adversity and worked hard to still produce a working product - even if it is not quite as polished as I hoped. I feel I have grown significantly not just academically but as a person; the project has helped improve my resilience, discipline and confidence as a problem solver and software developer.

The project gave me valuable experience working through a full software development cycle: conducting research, defining requirements, designing and developing the app and finally testing and evaluating. It has strengthened my problem-solving skills and taught me how breaking down a large problem into smaller, more manageable tasks which helps to not be overwhelmed.

Working with OpenAI's API was a new experience for me which I found to be an exciting challenge that kept me engaged in the project. Especially when motivation was low, the novelty of a new concept/technology was nice to have. Furthermore, I was eager to make use of its potential to reduce the workload for teachers as my mum works in education. I have seen first-hand how demanding the job can be so I knew I had the opportunity to build an app that could make a difference. The importance of prompt engineering surprised me. Prior to this project, I did not realise the importance of incorporating prompt engineering techniques and the impact the quality of a prompt can make on the output.

Overall, I believe the aim of the project has been achieved and I have produced a solution that could reduce the time spent by teachers on planning lessons. This project has taught me a lot about myself. I have improved my understanding of how best I personally learn new concepts. I have also learned how to deal with setbacks and work effectively under the pressure of a deadline while still meeting a goal. I have enjoyed making use of new technologies alongside giving me experience with every aspect of software development. It has also taught me how to design an application with a user in mind, and that it is okay to sometimes make changes from the original plan - making sacrifices is sometimes necessary if it means saving time while still meeting the main objectives. I feel this project was good at preparing me for the kind of work I will be doing in industry and the role I start after graduating.

References

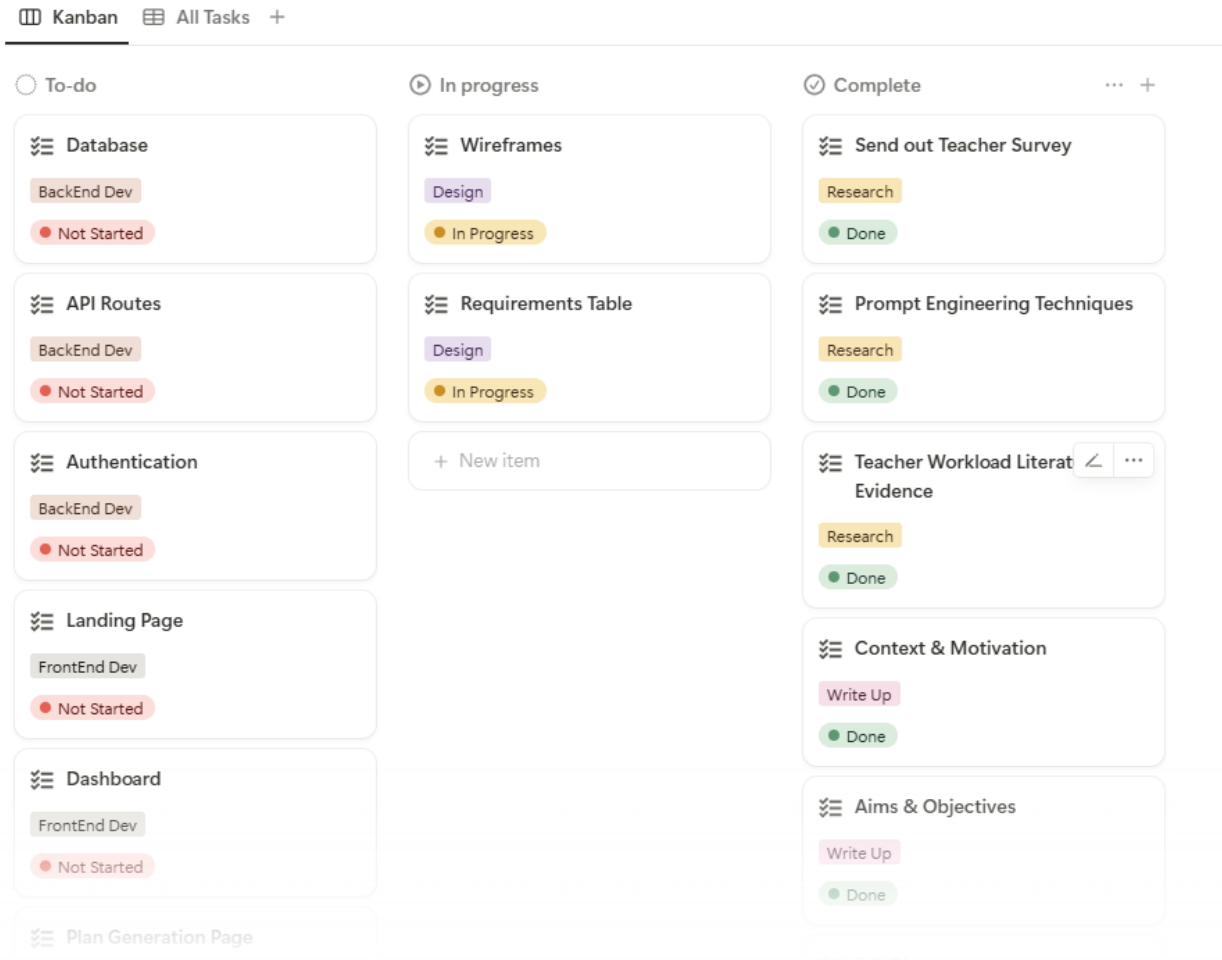
1. Ofsted, "Teacher Well-being at Work in Schools and Further Education Providers," Jul. 2019. [Online]. Available: https://assets.publishing.service.gov.uk/media/5fb41122e90e07208d0d5df1/Teacher_well-being_report_110719F.pdf [Accessed: 13-Feb-2025].
2. UK Parliament, "Teacher Recruitment, Training and Retention: Fifth Report of Session 2022–23," House of Commons Education Committee, Mar. 2023. [Online]. Available: <https://committees.parliament.uk/publications/44798/documents/222606/default/> [Accessed: 03-Mar-2025].
3. W. Powell and S. Courchesne, "Opportunities and risks involved in using ChatGPT to create first-grade science lesson plans," *PLOS ONE*, vol. 19, no. 6, Jun. 2024 [Online]. Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0305337> [Accessed: 26-Mar-2025]
4. M. R. Karaman and I. Goksu, "Are Lesson Plans Created by ChatGPT More Effective? An Experimental Study," *International Journal of Technology in Education (IJTE)*, vol. 7, no. 1, pp. 107–127, 2024. [Online]. Available: https://www.researchgate.net/publication/377964050_Are_Lesson_Plans_Created_by_Chat_GPT_More_Effective_An_Experimental_Study [Accessed: 26-Mar-2025]
5. K. Kostas, D. Pappas, J. Tsitsani, and K. Mpakola, "Tailoring Education with GenAI: A New Horizon in Lesson Planning," *arXiv preprint*, Mar. 2024. [Online]. Available: <https://arxiv.org/abs/2403.12071> [Accessed: 26-Mar-2025]
6. A. Corp and C. Revelle, "ChatGPT is Here to Stay: Using ChatGPT with Student Teachers for Lesson Planning," *The Texas Forum of Teacher Education*, 2022. [Online]. Available: <https://www.scribd.com/document/775659186/Forum-2023-Corp-and-Revelle> [Accessed: 26-Mar-2025]
7. UC Davis Library, "Prompt Engineering for Generative AI," 2024. [Online]. Available: <https://guides.library.ucdavis.edu/genai/prompt> [Accessed: 29-Mar-2025]
8. The GIANT Room, "GIANT AI Guide for Educators – Prompt Engineering," 2024. [Online]. Available: <https://www.thegiantroom.com/blog/2024/9/20/giant-ai-guide-for-educators-prompt-engineering> [Accessed: 29-Mar-2025]
9. A. Schulhoff et al., "Prompting Survey: Recent Advances and Research Frontiers," *arXiv preprint*, Jun. 2024. [Online]. Available: <https://arxiv.org/abs/2406.06608> [Accessed: 29-Mar-2025]
10. TeachMateAI, "Free Tools & AI-Powered Education," [Online]. Available: <https://teachmateai.com/free-tools>
11. Twinkl, "About Us," [Online]. Available: <https://www.twinkl.co.uk/about-us> [Accessed: 30-Mar-2025]
12. Brisk Teaching, "Free AI Tools for Educators," 2024. [Online]. Available: <https://www.briskteaching.com/> [Accessed: 30-Mar-2025]
13. A. Watt, *Database Design – 2nd Edition*. Open Textbook Library, University of Washington, 2014. [Online]. Available: <https://opentextbc.ca/dbdesign01/chapter/chapter-8-entity-relationship-model/> [Accessed: 02-Apr-2025]

14. M. Wisniewski, "How to Design a Good ER Diagram Layout," *Vertabelo Database Modeler*, 2021. [Online]. Available: <https://vertabelo.com/blog/vertabelo-tips-good-er-diagram-layout/> [Accessed: 02-Apr-2025]
15. Prisma, "Next.js + Prisma Quickstart," *Prisma Docs*, 2024. [Online]. Available: <https://www.prisma.io/docs/guides/nextjs> [Accessed: 04-Apr-2025]
16. Vercel, "Using PostgreSQL with Prisma in Next.js," *Vercel Guides*, 2024. [Online]. Available: <https://vercel.com/guides/nextjs-prisma-postgres> [Accessed: 04-Apr-2025]
17. Google, "Using OAuth 2.0 to Access Google APIs," *Google Identity Documentation*, 2024. [Online]. Available: <https://developers.google.com/identity/protocols/oauth2> [Accessed: 04-Apr-2025]
18. NextAuth.js, "Overview," *NextAuth Documentation*, 2024. [Online]. Available: <https://next-auth.js.org/getting-started/introduction> [Accessed: 04-Apr-2025]
19. Figma, "The Collaborative Interface Design Tool," *Figma*, 2024. [Online]. Available: <https://www.figma.com> [Accessed: 08-Apr-2025]
20. J. Nielsen, "10 Usability Heuristics for User Interface Design," Nielsen Norman Group, 1994. [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/> [Accessed: 01-May-2025].
21. R. Luckin, W. Holmes, M. Griffiths, and L. B. Forcier, "Intelligence Unleashed: An argument for AI in Education," Pearson Education, 2016. Available: <https://www.pearson.com/content/dam/one-dot-com/one-dot-com/global/Files/about-pearson/innovation/open-ideas/IntelligenceUnleashedSPANISH.pdf> [Accessed: 02-Apr-2025]

Appendices

Appendix A – Kanban Board Mid Project

≡ Dissertation Kanban Board



Appendix B – Crazy 8 Wireframe Sketches

Figure B1 - Landing & Dashboard

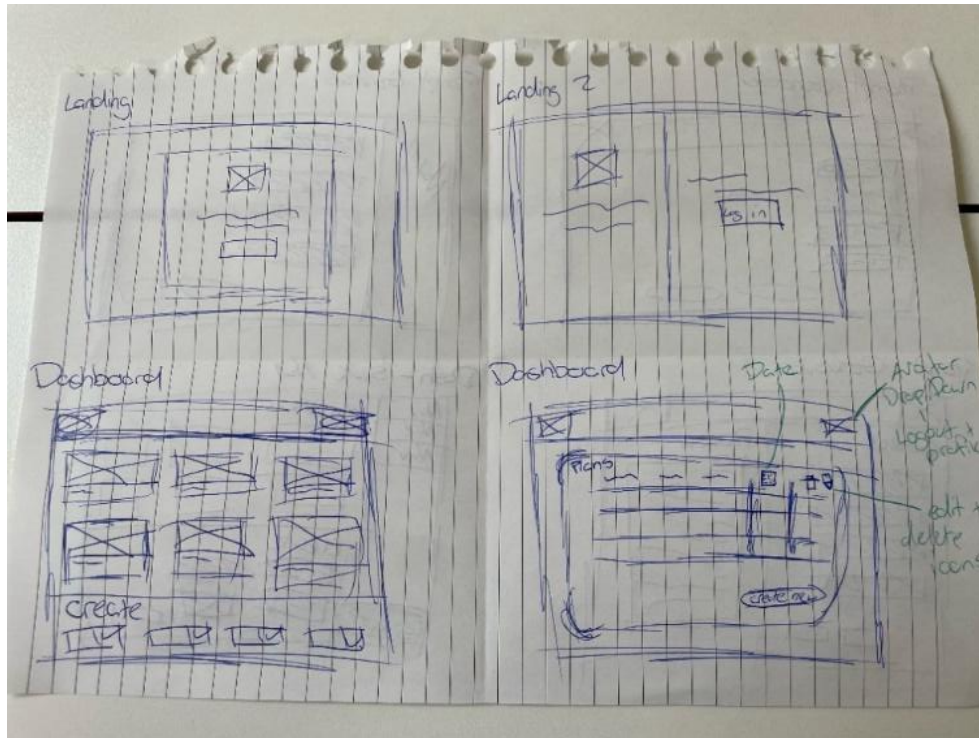
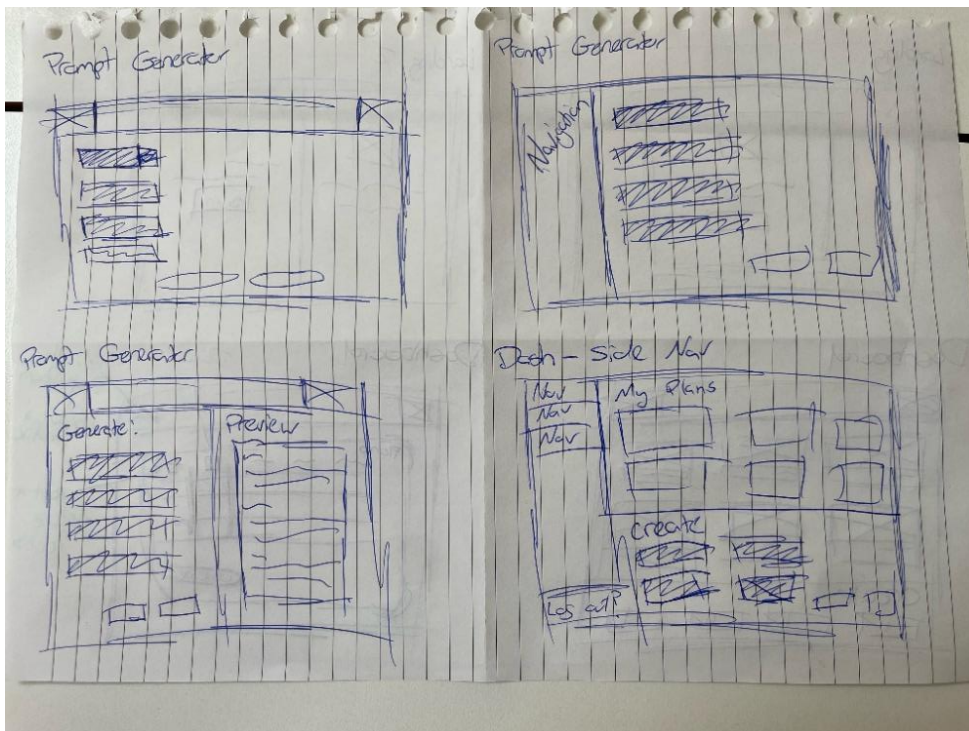
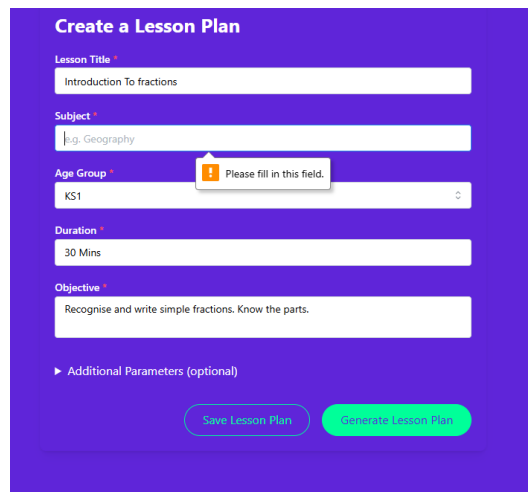


Figure B2 - Generator Ideas



Appendix C – Testing Screenshots

Figure C1 - Smooth error handling (Test T2):



Create a Lesson Plan

Lesson Title *
Introduction To fractions

Subject *
e.g. Geography

Age Group *
KS1

Duration *
30 Mins

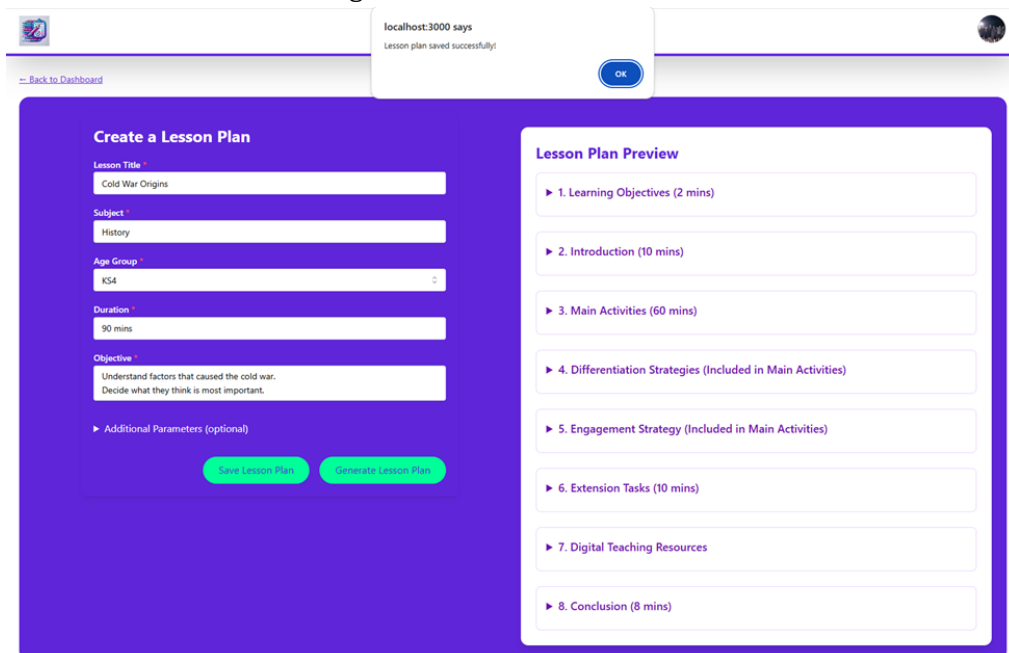
Objective *
Recognise and write simple fractions. Know the parts.

► Additional Parameters (optional)

Save Lesson Plan Generate Lesson Plan

Please fill in this field.

Figure C2 – Save confirmation message:



localhost:3000 says
Lesson plan saved successfully!

OK

← Back to Dashboard

Create a Lesson Plan

Lesson Title *
Cold War Origins

Subject *
History

Age Group *
KS4

Duration *
90 mins

Objective *
Understand factors that caused the cold war.
Decide what they think is most important.

► Additional Parameters (optional)

Save Lesson Plan Generate Lesson Plan

Lesson Plan Preview

- 1. Learning Objectives (2 mins)
- 2. Introduction (10 mins)
- 3. Main Activities (60 mins)
- 4. Differentiation Strategies (Included in Main Activities)
- 5. Engagement Strategy (Included in Main Activities)
- 6. Extension Tasks (10 mins)
- 7. Digital Teaching Resources
- 8. Conclusion (8 mins)

Figure C3 – Plans shown in raw database:

Tables

neondb public Search... Account LessonPlan Session User _prisma_migrations

2 rows • 181ms

id text	title text	content text	subject text	yearGroup text
cma0whr3l0001vpvsjbc3...	Muscle Pairs	<p>## 1. Learning Objectives (5 min...	PE	KS4
cma2ejdmn0001vpzwnodx...	Cold War Origins	## 1. Learning Objectives (2 mins) ...	History	KS4

Figure C4 – Successful Download of a file

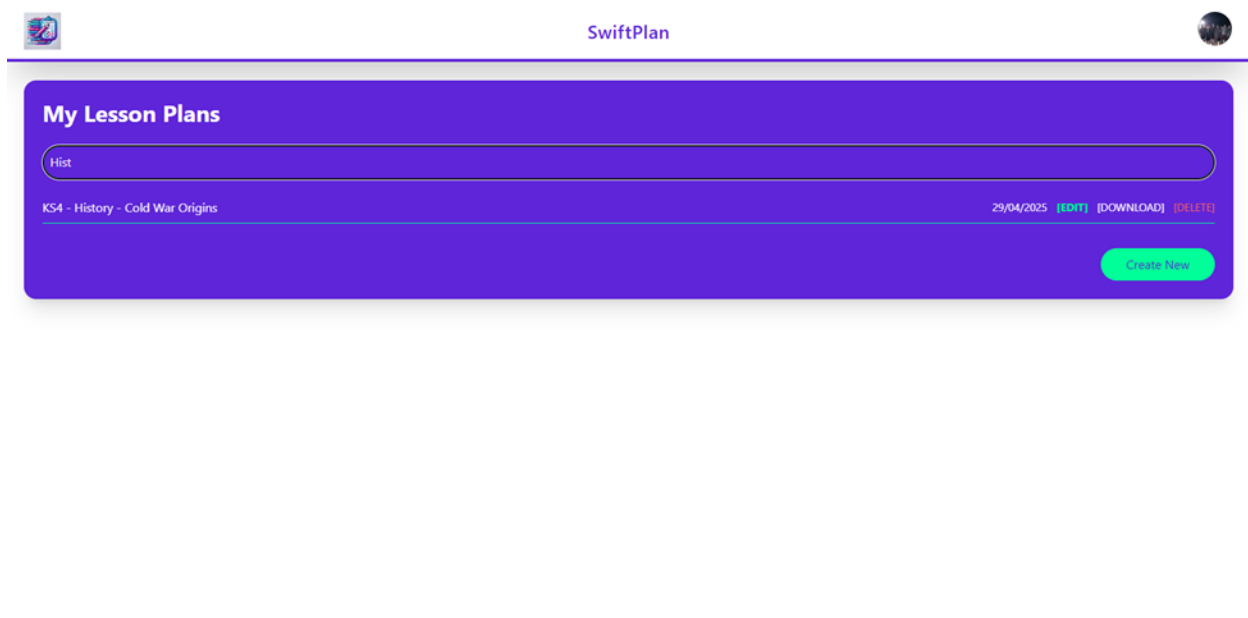
SwiftPlan

muscle-pairs (2).txt
2,949 B • Done

29/04/2025 [EDIT] [DOWNLOAD] [DELETE]

28/04/2025 [EDIT] [DOWNLOAD] [DELETE]

Figure C6: Search filter working correctly.



Appendix D – Downloaded files of generated plans

Figure D1 – A lesson planned for a physical education class with the topic of muscle pairs.

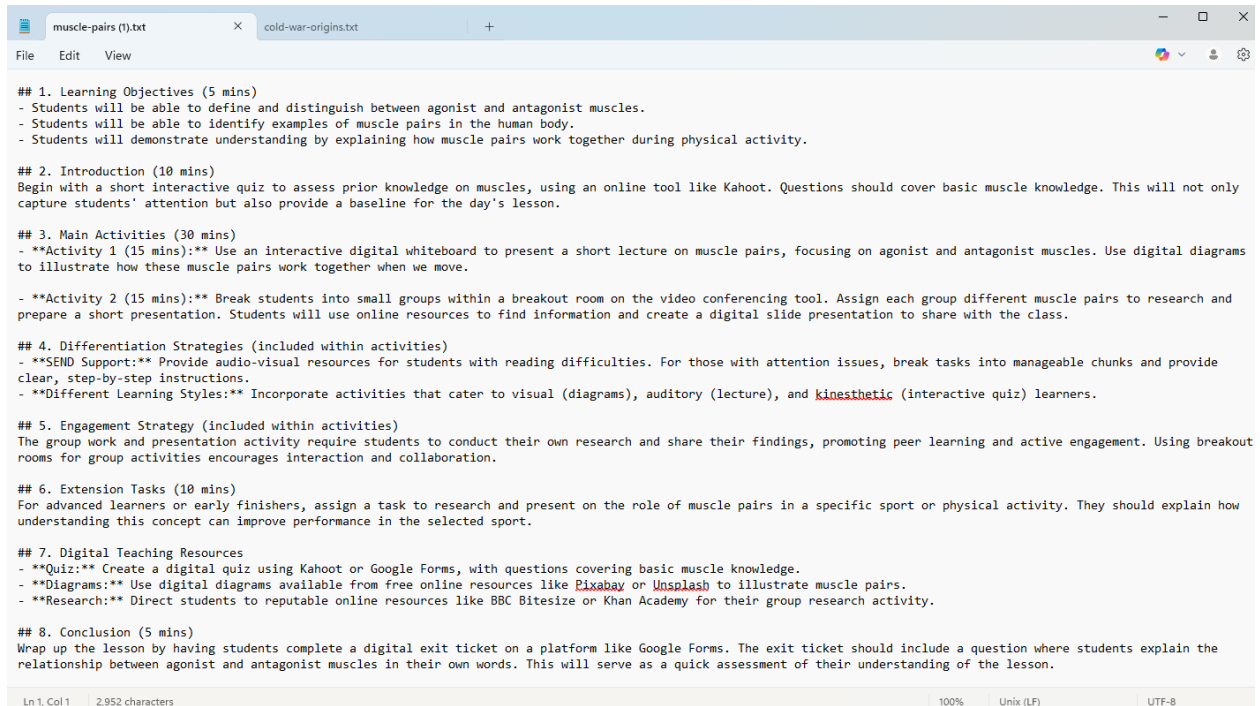
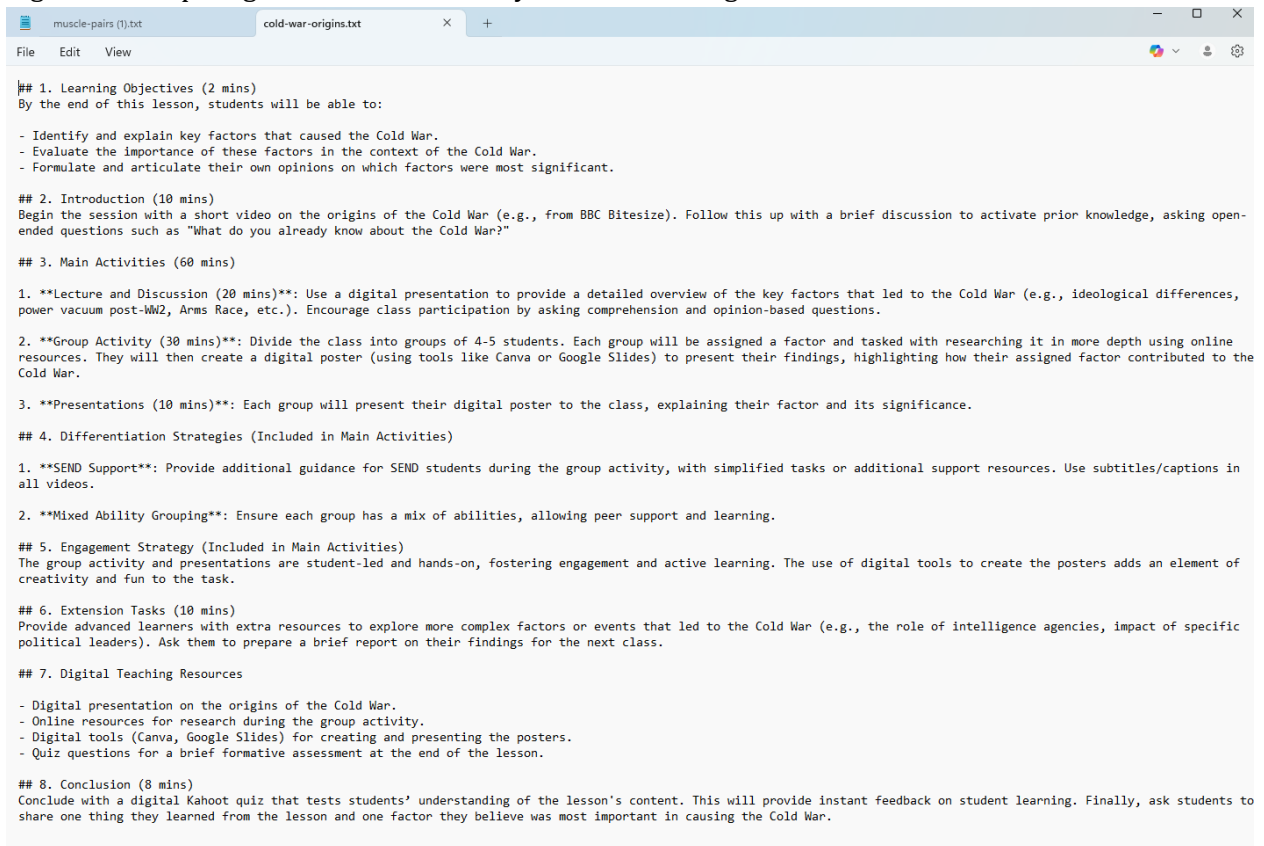


Figure D2 – A plan generated for a history class on the origins of the cold war



The image shows a screenshot of a text editor window with two tabs: "muscle-pairs (1).txt" and "cold-war-origins.txt". The "cold-war-origins.txt" tab is active, displaying a lesson plan. The plan is structured with numbered sections, each starting with a header like "## 1. Learning Objectives (2 mins)". The content includes learning objectives, an introduction with a video suggestion, main activities with sub-points for lecture, group work, and presentations, differentiation strategies, engagement strategies, extension tasks, digital teaching resources, and a conclusion. The text is formatted with bold markers for emphasis.

```
## 1. Learning Objectives (2 mins)
By the end of this lesson, students will be able to:

- Identify and explain key factors that caused the Cold War.
- Evaluate the importance of these factors in the context of the Cold War.
- Formulate and articulate their own opinions on which factors were most significant.

## 2. Introduction (10 mins)
Begin the session with a short video on the origins of the Cold War (e.g., from BBC Bitesize). Follow this up with a brief discussion to activate prior knowledge, asking open-ended questions such as "What do you already know about the Cold War?"

## 3. Main Activities (60 mins)

1. **Lecture and Discussion (20 mins)**: Use a digital presentation to provide a detailed overview of the key factors that led to the Cold War (e.g., ideological differences, power vacuum post-WW2, Arms Race, etc.). Encourage class participation by asking comprehension and opinion-based questions.

2. **Group Activity (30 mins)**: Divide the class into groups of 4-5 students. Each group will be assigned a factor and tasked with researching it in more depth using online resources. They will then create a digital poster (using tools like Canva or Google Slides) to present their findings, highlighting how their assigned factor contributed to the Cold War.

3. **Presentations (10 mins)**: Each group will present their digital poster to the class, explaining their factor and its significance.

## 4. Differentiation Strategies (Included in Main Activities)

1. **SEND Support**: Provide additional guidance for SEND students during the group activity, with simplified tasks or additional support resources. Use subtitles/captions in all videos.

2. **Mixed Ability Grouping**: Ensure each group has a mix of abilities, allowing peer support and learning.

## 5. Engagement Strategy (Included in Main Activities)
The group activity and presentations are student-led and hands-on, fostering engagement and active learning. The use of digital tools to create the posters adds an element of creativity and fun to the task.

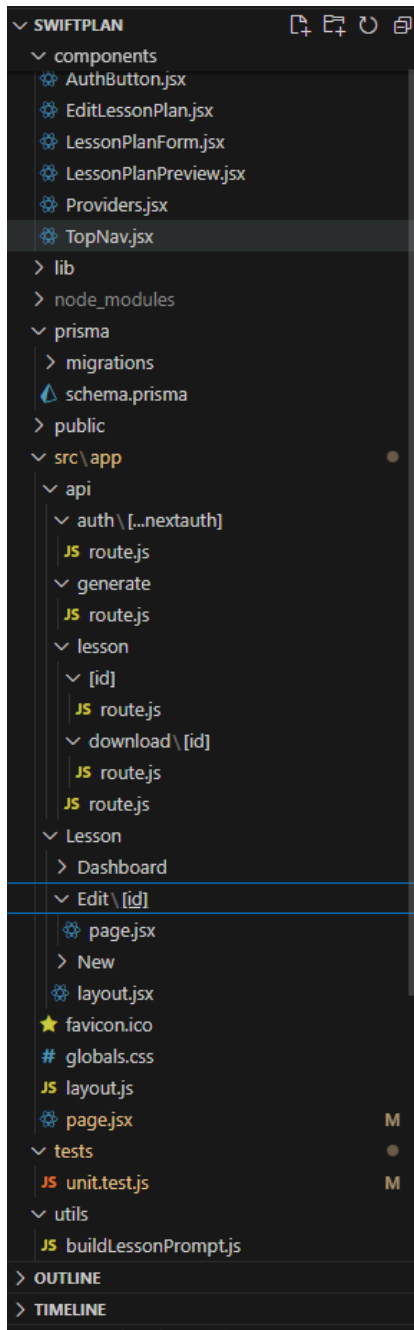
## 6. Extension Tasks (10 mins)
Provide advanced learners with extra resources to explore more complex factors or events that led to the Cold War (e.g., the role of intelligence agencies, impact of specific political leaders). Ask them to prepare a brief report on their findings for the next class.

## 7. Digital Teaching Resources

- Digital presentation on the origins of the Cold War.
- Online resources for research during the group activity.
- Digital tools (Canva, Google Slides) for creating and presenting the posters.
- Quiz questions for a brief formative assessment at the end of the lesson.

## 8. Conclusion (8 mins)
Conclude with a digital Kahoot quiz that tests students' understanding of the lesson's content. This will provide instant feedback on student learning. Finally, ask students to share one thing they learned from the lesson and one factor they believe was most important in causing the Cold War.
```

Appendix E – Folder Structure



Appendix F – Think Aloud Session Notes

Participant: Computer Science Student (pretending to be a secondary school teacher)

Date: 02/05/2025

Session type: Informal think-aloud

Scenario: “Imagine you are a secondary school teacher trying to reduce your planning workload. You’ve heard about this new app that helps generate and manage lesson plans. Try to use it to create, view, and edit a plan.

Tasks To Complete:

1. Log in via the landing page
2. Create a new lesson plan
3. Review the generated plan in the preview panel
4. Return to dashboard and confirm plan appear
5. Edit an existing plan
6. Download a plan

Observational Notes & Verbal Feedback

Task	Observation	Relevant heuristics
Logging in	Login was obvious. Using google felt familiar and I like how I do not have to create and remember another password.	Visibility Of System State, Recognition over Recall
Generator form	I like it all in one page and the form is clearly labelled. Error handling is nice too I deliberately entered something wrong and it prompted me in a nicely to fix it”	Aesthetic and Minimalist Design & Error Prevention
Plan generation	It felt slow but I liked how the preview box showed me it was loading so I knew that something was happening..	Visibility of System Status
Dashboard	It is easy to search plans, I am not overwhelmed by too much info - just their title. Buttons to manage plans are obvious, I like how delete is red.	Flexibility and efficiency of use, error prevention, Aesthetic and Minimalist Design
Edit screen	This screen feels clunky. I find it hard to locate which part of the plan I want to edit.	User Control and Freedom & Aesthetic and Minimalist Design

Download	This was easy to complete the button to download plans was obvious, I like how the name is relevant and the formatting of the file is nice	Flexibility and Efficiency of use, Recognition over Recall
----------	--	--

Follow-up Discussion Summary

- Liked the consistency, colour scheme, visual clarity of the user interface.
- Did feel a bit slow to generate the lesson plans
- Wanted more structure in the plan editor.
- Overall, could achieve their goal and it did not take a significant amount of thinking or cognitive load to do so, the flow of the program felt natural and obvious.

Appendix G – Performance Results

Figure G1 – POST request timing result in console

```
POST /api/generate 200 in 23412ms
```

Figure G2 – POST request timing result in console

```
POST /api/generate 200 in 23899ms
```

Figure G3 – POST request timing result in console

```
POST /api/generate 200 in 32615ms
```

Figure G4 – POST request timing result shown in Google DevTools

Request/Response		DURATION
Request sent		0.45 ms
Waiting for server response		32.62 s
Content Download		0.71 ms
<u>Explanation</u>		32.62 s